

CS1050 Computer Organization and Digital Design

Lab 9-10

Nano Processor Design

Student Name	Index Number
Madhushan G.W.P.	240403E
Munasinghe B.S.M.P.	240439T
Mayoorthanan S.	240426C
Musab A.	240441R

1. Lab Task

The objective of this project was to design and implement a 4-bit Nanoprocessor capable of executing a set of four instructions — MOVI, ADD, NEG, and JZR — entirely in VHDL, targeting the Basys3 FPGA board. The processor was constructed from individual sub-components including a Program Counter, PC Incrementer, Program ROM, Instruction Decoder, Register Bank, multiplexers, a 4-bit Add/Subtract unit, a Seven-Segment Display decoder, and a Slow Clock divider. Each component was designed, simulated, and then integrated into the top-level structural entity (Nano_processor). The design was verified through simulation before deployment on hardware.

2. Instruction Set

The Nanoprocessor supports four 12-bit instructions. The opcode occupies bits [11:9], the destination register Rd occupies bits [8:6], and the remaining bits carry operands or an immediate value.

Instruction	Description	12-bit Format
MOVI Rd, #imm	Load 4-bit immediate into register Rd	100 Rd[2:0] imm[3:0] 00
ADD Rd, Rs1, Rs2	$Rd \leftarrow Rs1 + Rs2$	001 Rd[2:0] Rs1[2:0] Rs2[2:0]
NEG Rd, Rs	$Rd \leftarrow 0 - Rs$ (2's complement negation)	011 Rd[2:0] Rs[2:0] 000
JZR Rs, addr	If $Rs == 0$, $PC \leftarrow addr$; else $PC \leftarrow PC+1$	010 Rs[2:0] addr[2:0] 000

Opcode encoding: ADD = 001, JZR = 010, NEG = 011, MOVI = 100. The NOP (opcode 000) is also implicitly supported — the decoder issues no control signals for unrecognized opcodes.

3. Assembly Program and Machine Code

The following program is stored in the Program ROM. It demonstrates MOVI, ADD, NEG, and JZR instructions, storing computed results in registers R2 through R6.

Addr	Assembly	Machine Code	Comment
000	MOVI R0, #5	100000010100	Load immediate 5 into R0
001	MOVI R1, #3	100001001100	Load immediate 3 into R1
010	ADD R2, R0, R1	001010000001	$R2 \leftarrow R0 + R1 = 8$
011	NEG R3, R1	011011001000	$R3 \leftarrow -R1 = -3$
100	ADD R4, R2, R3	001100010011	$R4 \leftarrow R2 + R3 = 5$
101	JZR R0, 7	010000111000	$R0 \neq 0$, so no jump (fall through)
110	MOVI R5, #1	100101000100	Load immediate 1 into R5
111	ADD R6, R0, R1	001110000001	$R6 \leftarrow R0 + R1 = 8$

Execution trace: R0 = 5, R1 = 3, R2 = 8, R3 = -3 (= 0xD in 4-bit two's complement), R4 = 5 (8 - 3), R5 = 1, R6 = 8. Since R0 = 5 $\neq$ 0, the JZR at address 101 does not branch.

4. Nanoprocessor Architecture

The processor is implemented as a single-cycle, synchronous design driven by a divided clock. Each rising edge of the slow clock causes the processor to fetch, decode, and execute one instruction. The data-path and control signals flow as follows:

- The Program Counter (PC) holds the 3-bit address of the current instruction.
- The PC Incrementer adds 1 to produce PC+1; the 2-to-1 3-bit MUX selects between PC+1 and a jump address based on the Jump enable signal.
- The ROM outputs the 12-bit instruction word addressed by the PC.
- The Instruction Decoder decodes the opcode and drives all control signals: Reg_En (8-bit one-hot), Mux_Sel_A/B, Imm_Val, Load_Imm, Add_Sub, Neg_En, Jump, and Jump_Addr.
- Two 8-to-1, 4-bit MUXes select the two ALU operands from the Register Bank. For NEG operations, ALU input A is forced to "0000" by a concurrent signal assignment controlled by Neg_En.
- The 4-bit Add/Subtract unit computes the result and asserts Overflow and Zero flags.
- A 2-to-1, 4-bit write-back MUX selects between the ALU result and the immediate value (controlled by Load_Imm), feeding the Register Bank input.

- The Register Bank contains eight 4-bit registers. Only the register indexed by Reg_En receives the new value on the next clock edge.
- For JZR, a combinational Zero_check signal (ALU_A_raw = 0) is ANDed with the Jump flag to form Jump_en, which steers the PC MUX.

The Seven-Segment Decoder and Slow Clock are peripheral components included in the project but instantiated outside the Nano_processor entity for board-level integration.

5. Component Descriptions

The table below summarises every VHDL module developed for this project.

Component	Function	Implementation
three_bit_adder	General-purpose 3-bit adder with carry-in and carry-out	Behavioral: uses NUMERIC_STD unsigned addition on a 4-bit temp signal
pc_incrementer	Adds 1 to the 3-bit PC address each cycle	Structural: wraps three_bit_adder with B fixed to "001" and Cin = '0'
Program_Counter	Holds and updates the 3-bit memory address of the next instruction	Synchronous D register with asynchronous active-high Reset; loads on rising edge when Load = '1'
ROM	Stores the 8-instruction program (12 bits each)	Behavioral case statement over 3-bit address; purely combinational
Instruction_decoder	Decodes 12-bit instruction and drives all control signals	Behavioral case on opcode[2:0]; generates Reg_En, Mux_Sel_A/B, Imm_Val, Load_Imm, Add_Sub, Neg_En, Jump, Jump_Addr
Reg_4bit	4-bit D flip-flop register with synchronous load and async reset	Behavioral: clocked process, Reset clears output; rising edge loads D when Load='1'
Registers_Bank	Bank of eight 4-bit registers (R0–R7) with individual enable lines	Structural: eight instances of Reg_4bit; Reg_En(i) drives each register's Load pin
Mux_2_3bit	2-to-1 multiplexer, 3-bit wide (selects PC+1 or jump address)	Behavioral: single concurrent signal assignment with when/else

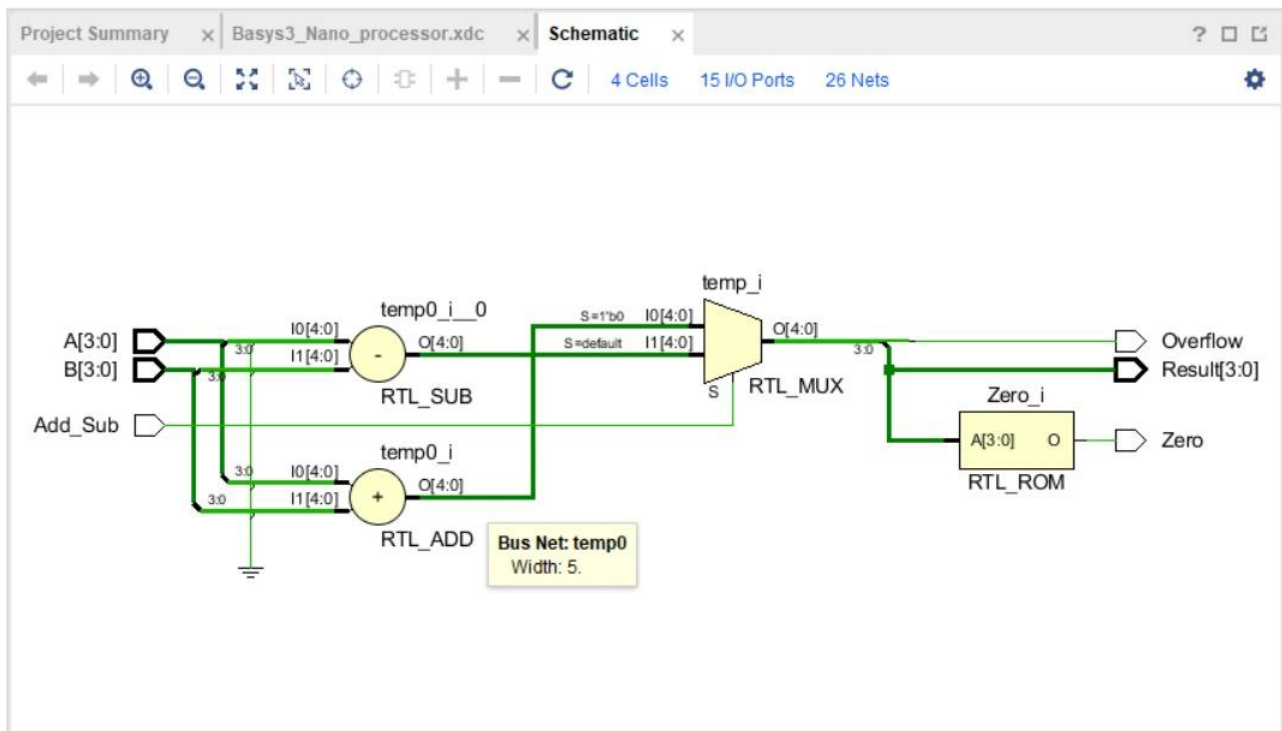
Mux_2_4bit	2-to-1 multiplexer, 4-bit wide (selects ALU result or immediate value for write-back)	Behavioral: single concurrent signal assignment with when/else
Mux_8_4bit	8-to-1 multiplexer, 4-bit wide (selects ALU input A from register bank)	Behavioral: selected signal assignment (with/select) over 3-bit Sel
Mux_8_4bit_new	8-to-1 multiplexer with enable, 4-bit wide (selects ALU input B; outputs 0000 when disabled)	Behavioral: selected signal assignment plus Enable gate; outputs "0000" when Enable='0'
Add_Sub_unit	4-bit arithmetic unit: adds or subtracts two operands; outputs Overflow and Zero flags	Behavioral: NUMERIC_STD unsigned add/subtract on 5-bit temp; Overflow = temp(4), Zero checks temp(3:0)
Tri_State_buffer	Single-bit tri-state buffer (outputs 'Z' when disabled)	Behavioral: concurrent assignment with 'Z' on disable
Tri_State_buffer_4bit	4-bit tri-state buffer (outputs "ZZZZ" when disabled)	Behavioral: concurrent assignment; used for bus isolation if required
Seven_Seg_Decoder	Converts 4-bit binary value to active-low 7-segment encoding for Basys3	Behavioral: case statement covering 0–9 (decimal); others outputs blank (1111111)
Slow_Clock	Divides 100 MHz board clock down to ~1 Hz for observable processor operation	Behavioral: counter toggles clk_reg every 50,000,000 cycles; Reset clears counter and output
Nano_processor	Top-level structural entity integrating all sub-components	Structural: instantiates PC, pc_incrementer, Mux_2_3bit, ROM, Instruction_decoder, Registers_Bank, two Mux_8_4bit instances, Add_Sub_unit, Mux_2_4bit

6. VHDL Source Files

All design source files are listed below. Full source code is provided; simulation testbench files and timing diagrams are to be inserted in the relevant subsections.

3bit_adder.vhd

Schematic Diagram



Design Source Code

```
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:33:05 PM
-- Design Name:
-- Module Name: 3bit_adder - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
```

```
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
--
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

use IEEE.NUMERIC_STD.ALL;

entity three_bit_adder is
    Port ( A      : in  STD_LOGIC_VECTOR(2 downto 0);
          B      : in  STD_LOGIC_VECTOR(2 downto 0);
          Cin     : in  STD_LOGIC;
          Sum     : out STD_LOGIC_VECTOR(2 downto 0);
          Cout    : out STD_LOGIC);
end three_bit_adder ;

architecture Behavioral of three_bit_adder is
    signal temp : STD_LOGIC_VECTOR(3 downto 0);
begin
    temp <= std_logic_vector(
        unsigned('0' & A) +
        unsigned('0' & B) +
        ("000" & Cin));
    Sum  <= temp(2 downto 0);
    Cout <= temp(3);
end Behavioral;
```

Simulation Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_three_bit_adder is
end tb_three_bit_adder;
```

architecture Behavioral of tb_three_bit_adder is

```
component three_bit_adder
  Port ( A      : in  STD_LOGIC_VECTOR(2 downto 0);
        B      : in  STD_LOGIC_VECTOR(2 downto 0);
        Cin     : in  STD_LOGIC;
        Sum     : out STD_LOGIC_VECTOR(2 downto 0);
        Cout    : out STD_LOGIC);
end component;

signal A, B : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Cin  : STD_LOGIC := '0';
signal Sum  : STD_LOGIC_VECTOR(2 downto 0);
signal Cout : STD_LOGIC;
```

begin

```
DUT: three_bit_adder port map(A=>A, B=>B, Cin=>Cin, Sum=>Sum, Cout=>Cout);
```

```
process
```

```
begin
```

```
-- Test 1: 0 + 0 + 0 = 0, Cout=0
A <= "000"; B <= "000"; Cin <= '0'; wait for 20 ns;
assert Sum = "000" and Cout = '0'
  report "FAIL: 0+0+0" severity error;

-- Test 2: 1 + 1 + 0 = 2, Cout=0
A <= "001"; B <= "001"; Cin <= '0'; wait for 20 ns;
assert Sum = "010" and Cout = '0'
  report "FAIL: 1+1+0" severity error;

-- Test 3: 3 + 4 + 0 = 7, Cout=0
A <= "011"; B <= "100"; Cin <= '0'; wait for 20 ns;
assert Sum = "111" and Cout = '0'
  report "FAIL: 3+4+0" severity error;

-- Test 4: 5 + 3 + 0 = 8 -> Sum=0, Cout=1
A <= "101"; B <= "011"; Cin <= '0'; wait for 20 ns;
assert Sum = "000" and Cout = '1'
  report "FAIL: 5+3+0 overflow" severity error;

-- Test 5: 7 + 7 + 1 = 15 -> Sum=7, Cout=1
A <= "111"; B <= "111"; Cin <= '1'; wait for 20 ns;
assert Sum = "111" and Cout = '1'
  report "FAIL: 7+7+1" severity error;

-- Test 6: Cin propagation: 3 + 3 + 1 = 7
A <= "011"; B <= "011"; Cin <= '1'; wait for 20 ns;
assert Sum = "111" and Cout = '0'
  report "FAIL: 3+3+1" severity error;

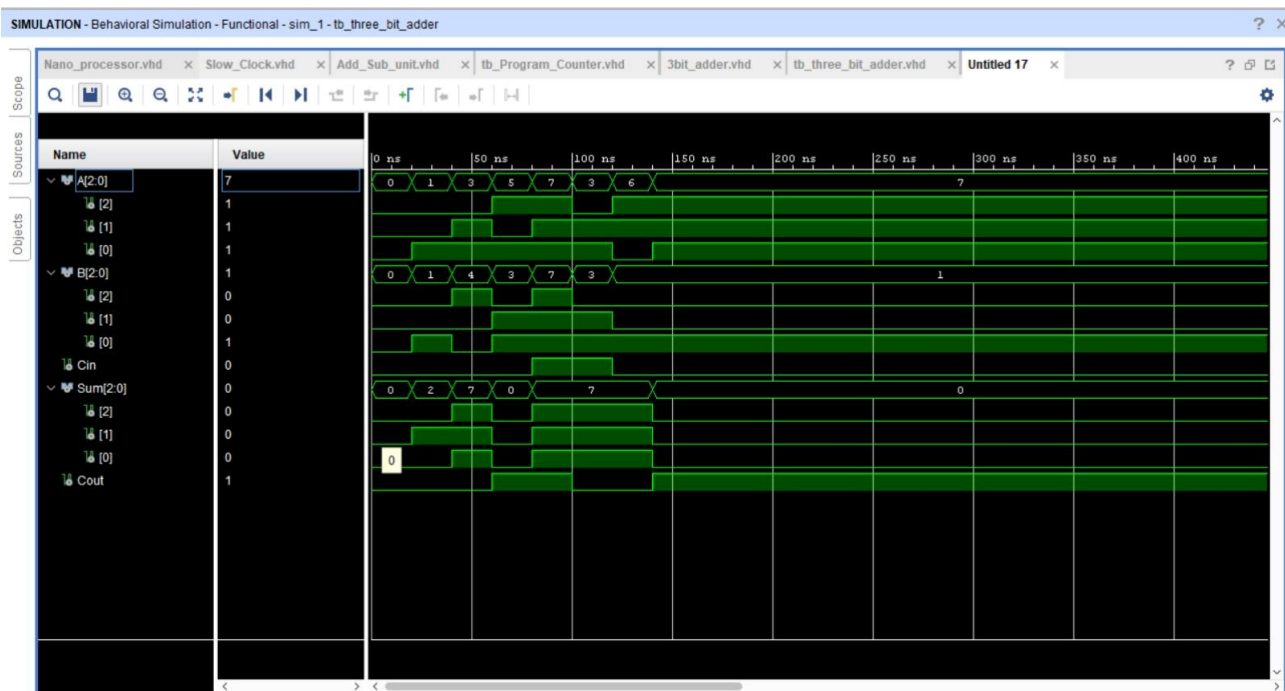
-- Test 7: Incrementer use-case: 6 + 1 + 0 = 7
A <= "110"; B <= "001"; Cin <= '0'; wait for 20 ns;
assert Sum = "111" and Cout = '0'
  report "FAIL: 6+1+0" severity error;

-- Test 8: Wrap-around: 7 + 1 + 0 = 0 carry
A <= "111"; B <= "001"; Cin <= '0'; wait for 20 ns;
assert Sum = "000" and Cout = '1'
  report "FAIL: 7+1 wrap" severity error;

report "tb_three_bit_adder: ALL TESTS DONE" severity note;
wait;
end process;
```

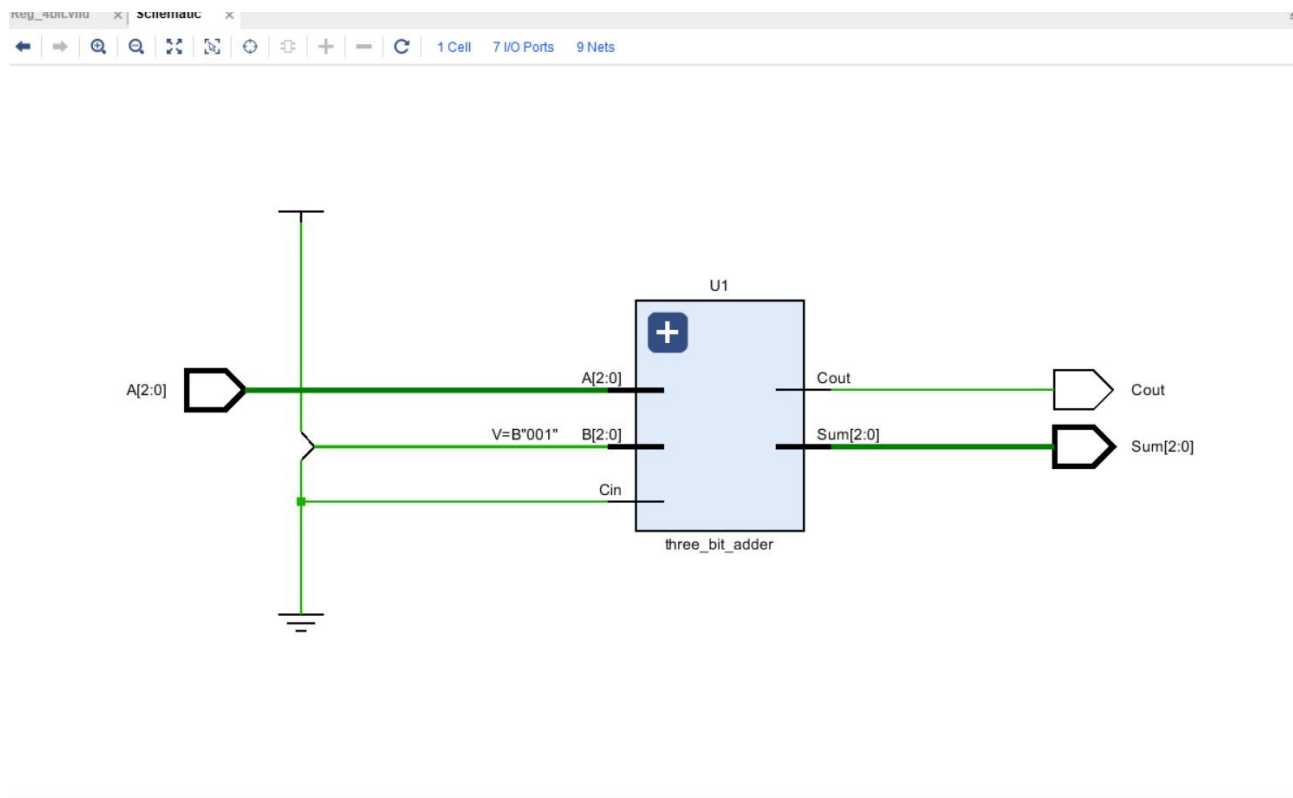
end Behavioral;

Timing Diagram



Adder 3bit.vhd (pc incrementer)

Schematic Diagram



Design Source Code

```
-----
--
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:11:38 PM
-- Design Name:
-- Module Name: adder_3bit - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
--

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity pc_incrementer is
    Port ( A      : in  STD_LOGIC_VECTOR(2 downto 0);
          Sum     : out STD_LOGIC_VECTOR(2 downto 0);
          Cout    : out STD_LOGIC);
end pc_incrementer ;

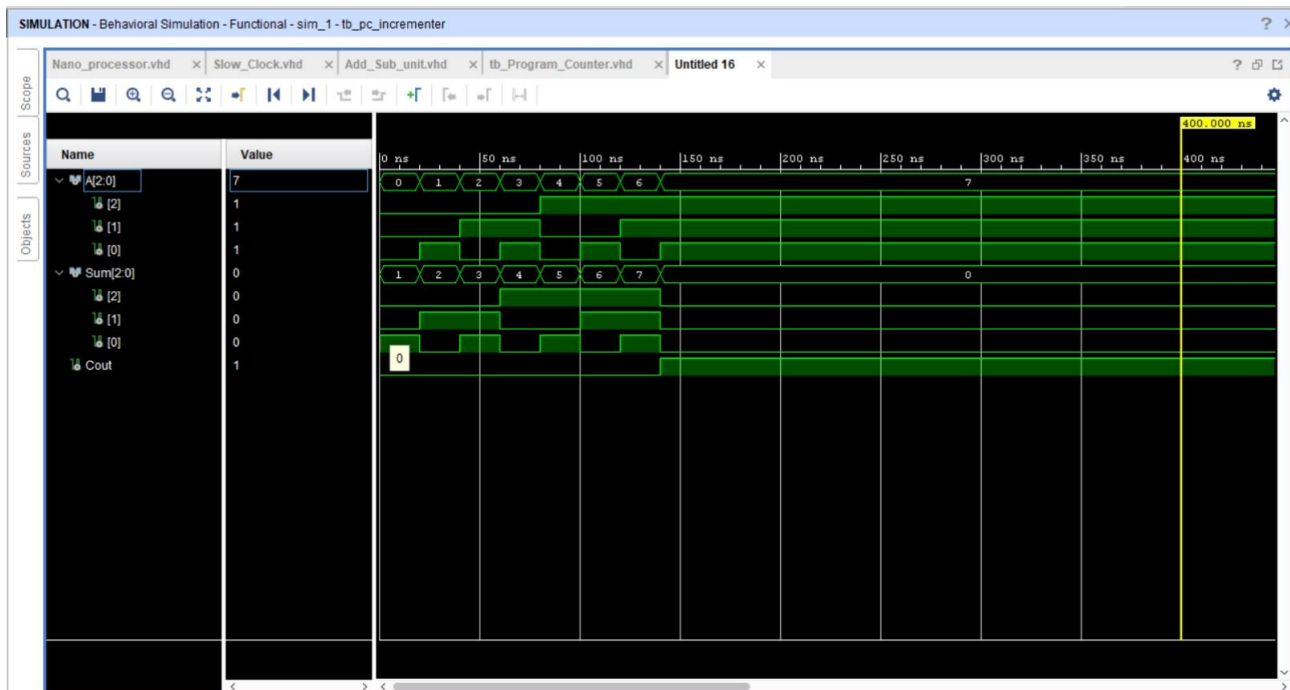
architecture Structural of pc_incrementer is
    component three_bit_adder
        Port ( A      : in  STD_LOGIC_VECTOR(2 downto 0);
              B      : in  STD_LOGIC_VECTOR(2 downto 0);
              Cin     : in  STD_LOGIC;
              Sum     : out STD_LOGIC_VECTOR(2 downto 0);
              Cout    : out STD_LOGIC);
    end component;
begin
    -- Adds 1 to PC: B = "001", Cin = '0'
    U1: three_bit_adder
        port map (A      => A,
                  B      => "001",
                  Cin    => '0',
                  Sum    => Sum,
                  Cout   => Cout);
end Structural;
```

Simulation Source Code

```
-----  
--  
-- Testbench : tb_pc_incrementer  
-- DUT       : pc_incrementer  
-- Tests     : +1 on all 3-bit values including wrap-around  
-----  
--  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity tb_pc_incrementer is  
end tb_pc_incrementer;  
  
architecture Behavioral of tb_pc_incrementer is  
  
    component pc_incrementer  
        Port ( A      : in  STD_LOGIC_VECTOR(2 downto 0);  
              Sum     : out STD_LOGIC_VECTOR(2 downto 0);  
              Cout    : out STD_LOGIC);  
    end component;  
  
    signal A      : STD_LOGIC_VECTOR(2 downto 0) := "000";  
    signal Sum     : STD_LOGIC_VECTOR(2 downto 0);  
    signal Cout    : STD_LOGIC;  
  
begin  
  
    DUT: pc_incrementer port map(A=>A, Sum=>Sum, Cout=>Cout);  
  
    process  
    begin  
        -- Test all 8 PC values  
        A <= "000"; wait for 20 ns;  -- 0+1=1, Cout=0  
        assert Sum = "001" and Cout = '0' report "FAIL: PC=0" severity error;  
  
        A <= "001"; wait for 20 ns;  -- 1+1=2  
        assert Sum = "010" and Cout = '0' report "FAIL: PC=1" severity error;  
  
        A <= "010"; wait for 20 ns;  -- 2+1=3  
        assert Sum = "011" and Cout = '0' report "FAIL: PC=2" severity error;  
  
        A <= "011"; wait for 20 ns;  -- 3+1=4  
        assert Sum = "100" and Cout = '0' report "FAIL: PC=3" severity error;  
  
        A <= "100"; wait for 20 ns;  -- 4+1=5  
        assert Sum = "101" and Cout = '0' report "FAIL: PC=4" severity error;  
  
        A <= "101"; wait for 20 ns;  -- 5+1=6  
        assert Sum = "110" and Cout = '0' report "FAIL: PC=5" severity error;  
  
        A <= "110"; wait for 20 ns;  -- 6+1=7  
        assert Sum = "111" and Cout = '0' report "FAIL: PC=6" severity error;  
  
        A <= "111"; wait for 20 ns;  -- 7+1=8 â†’ Sum=0, Cout=1 (wrap)  
        assert Sum = "000" and Cout = '1' report "FAIL: PC=7 wrap" severity  
error;  
  
        report "tb_pc_incrementer: ALL TESTS DONE" severity note;  
        wait;  
    end process;  
end process;
```

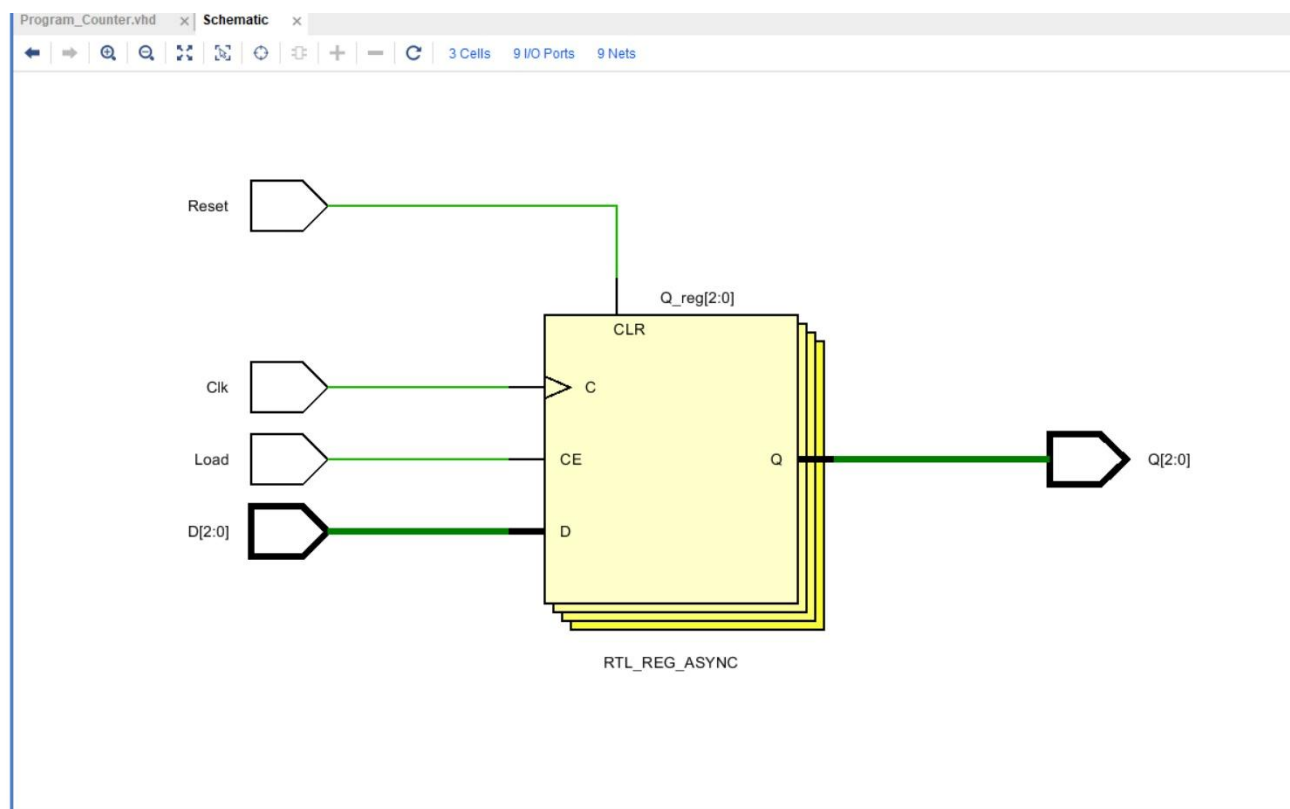
```
end Behavioral;
```

Timing Diagram



Program Counter.vhd

Schematic Diagram



Design Source Code

```
-----
--
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:40:59 PM
-- Design Name:
-- Module Name: Program_Counter - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Program_Counter is
    Port ( D      : in  STD_LOGIC_VECTOR(2 downto 0);
          Load   : in  STD_LOGIC;
          Reset   : in  STD_LOGIC;
          Clk     : in  STD_LOGIC;
          Q       : out STD_LOGIC_VECTOR(2 downto 0));
end Program_Counter;

architecture Behavioral of Program_Counter is
begin
    process(Clk, Reset)
    begin
        if Reset = '1' then
            Q <= "000";
        elsif rising_edge(Clk) then
            if Load = '1' then
                Q <= D;
            end if;
        end if;
    end process;
end Behavioral;
```

Simulation Source Code

```
-----
--
-- Testbench : tb_Program_Counter
-- DUT       : Program_Counter
-- Tests     : async reset, load on rising edge, hold when Load=0
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Program_Counter is
end tb_Program_Counter;

architecture Behavioral of tb_Program_Counter is

    component Program_Counter
        Port ( D      : in  STD_LOGIC_VECTOR(2 downto 0);
              Load   : in  STD_LOGIC;
              Reset   : in  STD_LOGIC;
              Clk     : in  STD_LOGIC;
              Q       : out STD_LOGIC_VECTOR(2 downto 0));
    end component;

    signal D      : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal Load   : STD_LOGIC := '0';
    signal Reset  : STD_LOGIC := '0';
    signal Clk    : STD_LOGIC := '0';
    signal Q      : STD_LOGIC_VECTOR(2 downto 0);

    constant CLK_PERIOD : time := 20 ns;

begin

    DUT: Program_Counter port map(D=>D, Load=>Load, Reset=>Reset, Clk=>Clk,
    Q=>Q);

    -- Clock generator
    Clk <= not Clk after CLK_PERIOD / 2;

    process
    begin
        -- Test 1: Async Reset
        Reset <= '1'; wait for 15 ns;
        assert Q = "000" report "FAIL: Reset did not clear Q" severity error;
        Reset <= '0'; wait for 5 ns;

        -- Test 2: Load D = 5 on rising edge
        D <= "101"; Load <= '1';
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "101" report "FAIL: Load 5" severity error;

        -- Test 3: Load D = 3
        D <= "011";
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "011" report "FAIL: Load 3" severity error;

        -- Test 4: Load = 0 & Q holds value
        D <= "111"; Load <= '0';
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "011" report "FAIL: Hold when Load=0" severity error;
```

```

-- Test 5: Reset overrides load
D <= "110"; Load <= '1'; Reset <= '1'; wait for 5 ns;
assert Q = "000" report "FAIL: Reset priority" severity error;
Reset <= '0';

-- Test 6: Load D = 7 after reset
D <= "111"; Load <= '1';
wait until rising_edge(Clk); wait for 5 ns;
assert Q = "111" report "FAIL: Load 7 after reset" severity error;

report "tb_Program_Counter: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

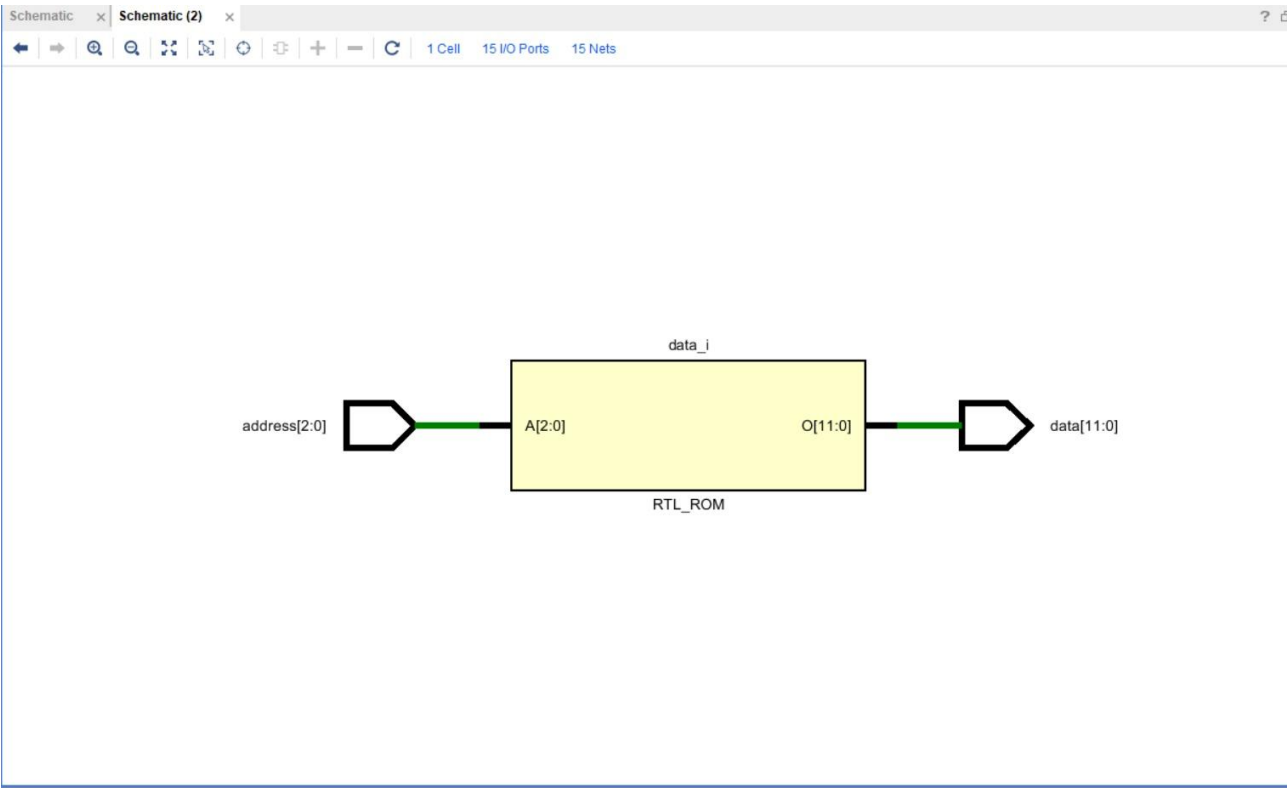
```

Timing Diagram



ROM.vhd

Schematic Diagram



Design Source Code

```
-----  
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:42:22 PM  
-- Design Name:  
-- Module Name: ROM - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity ROM is
    Port ( address : in  STD_LOGIC_VECTOR(2 downto 0);
          data      : out STD_LOGIC_VECTOR(11 downto 0));
end ROM;

architecture Behavioral of ROM is
begin
    process(address)
    begin
        case address is
            -- MOVI R0, #5 => 100 000 0101 00
            when "000" => data <= "100000010100";
            -- MOVI R1, #3 => 100 001 0011 00
            when "001" => data <= "100001001100";
            -- ADD R2,R0,R1 => 001 010 000 001
            when "010" => data <= "001010000001";
            -- NEG R3, R1  => 011 011 001 000
            when "011" => data <= "011011001000";
            -- ADD R4,R2,R3 => 001 100 010 011
            when "100" => data <= "001100010011";
            -- JZR R0, 7   => 010 000 111 000
            when "101" => data <= "010000111000";
            -- MOVI R5, #1 => 100 101 0001 00
            when "110" => data <= "100101000100";
            -- ADD R6,R0,R1 => 001 110 000 001
            when "111" => data <= "001110000001";
            when others => data <= "000000000000";
        end case;
    end process;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_ROM
-- DUT       : ROM
-- Tests     : all 8 addresses return exactly the expected 12-bit instruction
--
-- ROM program (from source):
--   addr 0: MOVI R0,#5  = "100000010100"
--   addr 1: MOVI R1,#3  = "100001001100"
--   addr 2: ADD R2,R0,R1= "001010000001"
--   addr 3: NEG R3,R1   = "011011001000"
--   addr 4: ADD R4,R2,R3= "001100010011"
--   addr 5: JZR R0,7    = "010000111000"
--   addr 6: MOVI R5,#1  = "100101000100"
--   addr 7: ADD R6,R0,R1= "001110000001"
--
-----
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```



```

entity tb_ROM is
end tb_ROM;

architecture Behavioral of tb_ROM is

    component ROM
        Port ( address : in  STD_LOGIC_VECTOR(2 downto 0);
              data      : out STD_LOGIC_VECTOR(11 downto 0));
    end component;

    signal address : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal data     : STD_LOGIC_VECTOR(11 downto 0);

begin

    DUT: ROM port map(address=>address, data=>data);

    process
    begin
        address <= "000"; wait for 20 ns;
        assert data = "100000010100"
            report "FAIL: addr 0 (MOVI R0,#5)" severity error;

        address <= "001"; wait for 20 ns;
        assert data = "100001001100"
            report "FAIL: addr 1 (MOVI R1,#3)" severity error;

        address <= "010"; wait for 20 ns;
        assert data = "001010000001"
            report "FAIL: addr 2 (ADD R2,R0,R1)" severity error;

        address <= "011"; wait for 20 ns;
        assert data = "011011001000"
            report "FAIL: addr 3 (NEG R3,R1)" severity error;

        address <= "100"; wait for 20 ns;
        assert data = "001100010011"
            report "FAIL: addr 4 (ADD R4,R2,R3)" severity error;

        address <= "101"; wait for 20 ns;
        assert data = "010000111000"
            report "FAIL: addr 5 (JZR R0,7)" severity error;

        address <= "110"; wait for 20 ns;
        assert data = "100101000100"
            report "FAIL: addr 6 (MOVI R5,#1)" severity error;

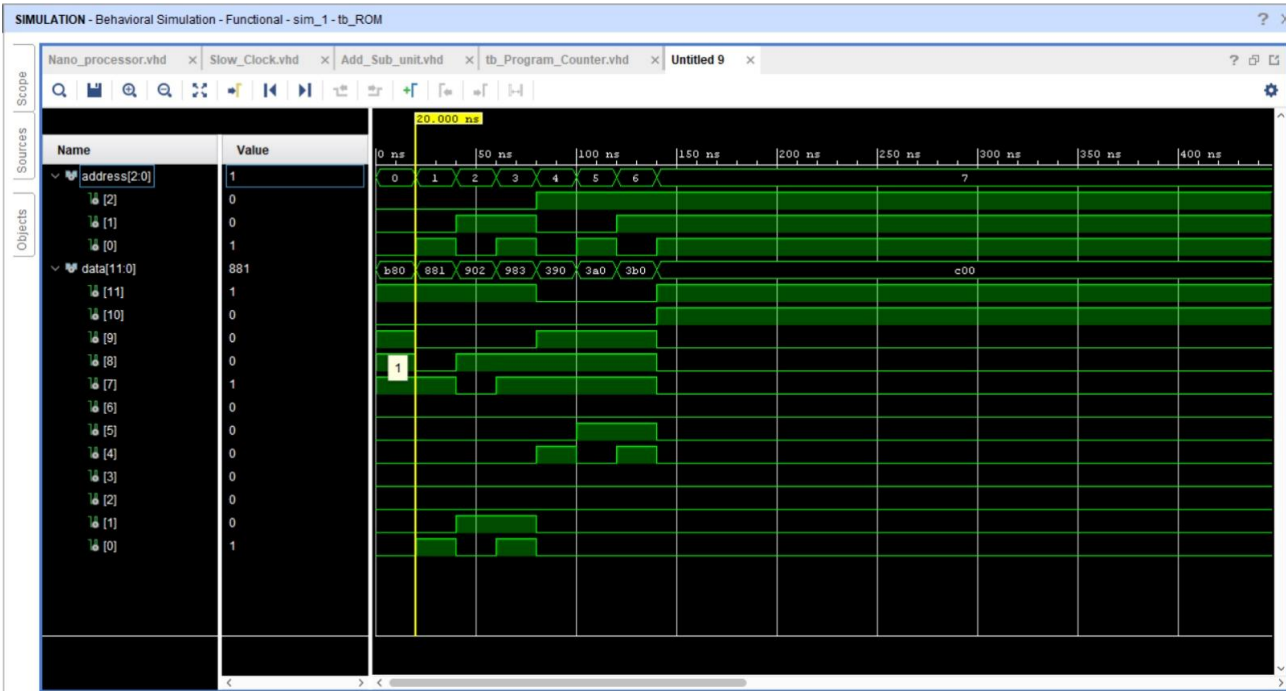
        address <= "111"; wait for 20 ns;
        assert data = "001110000001"
            report "FAIL: addr 7 (ADD R6,R0,R1)" severity error;

        report "tb_ROM: ALL TESTS DONE" severity note;
        wait;
    end process;

end Behavioral;

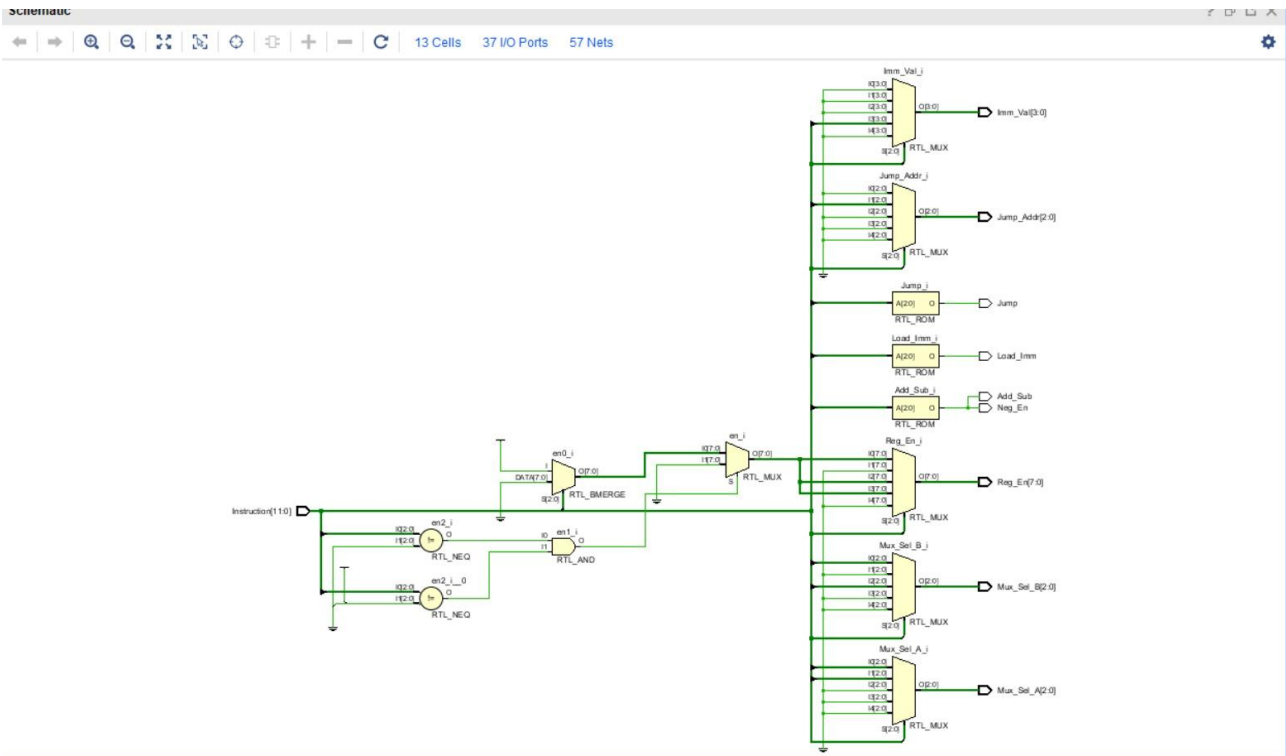
```

Timing Diagram



Instruction decoder.vhd

Schematic Diagram



Design Source Code

```
-----
--
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:43:08 PM
-- Design Name:
-- Module Name: Instruction_decoder - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
--

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
use IEEE.NUMERIC_STD.ALL;

entity Instruction_decoder is
    Port ( Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
          Reg_En       : out STD_LOGIC_VECTOR(7 downto 0);
          Mux_Sel_A    : out STD_LOGIC_VECTOR(2 downto 0);
          Mux_Sel_B    : out STD_LOGIC_VECTOR(2 downto 0);
          Imm_Val       : out STD_LOGIC_VECTOR(3 downto 0);
          Load_Imm      : out STD_LOGIC;
          Add_Sub       : out STD_LOGIC;
          Neg_En        : out STD_LOGIC;
          Jump          : out STD_LOGIC;
          Jump_Addr     : out STD_LOGIC_VECTOR(2 downto 0));
end Instruction_decoder;

architecture Behavioral of Instruction_decoder is
    signal opcode : STD_LOGIC_VECTOR(2 downto 0);
    signal Rd      : STD_LOGIC_VECTOR(2 downto 0);
begin
    opcode <= Instruction(11 downto 9);
    Rd     <= Instruction(8  downto 6);

    process(Instruction, opcode, Rd)
        variable en : STD_LOGIC_VECTOR(7 downto 0);
```

```

begin
    en := "000000000";
    if opcode /= "000" and opcode /= "010" then
        en(to_integer(unsigned(Rd))) := '1';
    end if;

    Reg_En    <= "000000000";
    Mux_Sel_A <= "000"; Mux_Sel_B <= "000";
    Imm_Val   <= "0000"; Load_Imm <= '0';
    Add_Sub   <= '0';   Neg_En    <= '0';
    Jump      <= '0';   Jump_Addr <= "000";

    case opcode is
        when "001" => -- ADD Rd, Rs1, Rs2
            Reg_En    <= en;
            Mux_Sel_A <= Instruction(5 downto 3);
            Mux_Sel_B <= Instruction(2 downto 0);
            Add_Sub   <= '0';
        when "010" => -- JZR Rs, addr
            Mux_Sel_A <= Rd;
            Jump      <= '1';
            Jump_Addr <= Instruction(5 downto 3);
        when "011" => -- NEG Rd, Rs (Rd = 0 - Rs)
            Reg_En    <= en;
            Mux_Sel_B <= Instruction(5 downto 3);
            Add_Sub   <= '1';
            Neg_En    <= '1'; -- force ALU_A = 0000
        when "100" => -- MOVI Rd, #imm
            Reg_En    <= en;
            Imm_Val   <= Instruction(5 downto 2);
            Load_Imm <= '1';
        when others => null; -- NOP
    end case;
end process;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Instruction_decoder
-- DUT       : Instruction_decoder
--
-- Instruction format:
--   bits [11:9] = opcode  bits [8:6] = Rd  bits [5:3] = Rs1  bits [2:0] = Rs2
--
-- Opcodes:
--   001 = ADD    Reg_En[Rd]=1, Mux_Sel_A=Rs1, Mux_Sel_B=Rs2, Add_Sub=0
--   010 = JZR    Mux_Sel_A=Rd, Jump=1, Jump_Addr=Rs1
--   011 = NEG    Reg_En[Rd]=1, Mux_Sel_B=Rs1, Add_Sub=1, Neg_En=1
--   100 = MOVI   Reg_En[Rd]=1, Imm_Val=bits[5:2], Load_Imm=1
--   000 = NOP    all outputs default/zero
--
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Instruction_decoder is
end tb_Instruction_decoder;

```

architecture Behavioral of tb_Instruction_decoder is

```

component Instruction_decoder
  Port ( Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
        Reg_En      : out STD_LOGIC_VECTOR(7 downto 0);
        Mux_Sel_A   : out STD_LOGIC_VECTOR(2 downto 0);
        Mux_Sel_B   : out STD_LOGIC_VECTOR(2 downto 0);
        Imm_Val     : out STD_LOGIC_VECTOR(3 downto 0);
        Load_Imm    : out STD_LOGIC;
        Add_Sub     : out STD_LOGIC;
        Neg_En      : out STD_LOGIC;
        Jump        : out STD_LOGIC;
        Jump_Addr   : out STD_LOGIC_VECTOR(2 downto 0));
end component;

signal Instruction : STD_LOGIC_VECTOR(11 downto 0) := "000000000000";
signal Reg_En      : STD_LOGIC_VECTOR(7 downto 0);
signal Mux_Sel_A   : STD_LOGIC_VECTOR(2 downto 0);
signal Mux_Sel_B   : STD_LOGIC_VECTOR(2 downto 0);
signal Imm_Val     : STD_LOGIC_VECTOR(3 downto 0);
signal Load_Imm    : STD_LOGIC;
signal Add_Sub     : STD_LOGIC;
signal Neg_En      : STD_LOGIC;
signal Jump        : STD_LOGIC;
signal Jump_Addr   : STD_LOGIC_VECTOR(2 downto 0);

```

begin

```

DUT: Instruction_decoder port map(
  Instruction => Instruction,
  Reg_En      => Reg_En,
  Mux_Sel_A   => Mux_Sel_A,
  Mux_Sel_B   => Mux_Sel_B,
  Imm_Val     => Imm_Val,
  Load_Imm    => Load_Imm,
  Add_Sub     => Add_Sub,
  Neg_En      => Neg_En,
  Jump        => Jump,
  Jump_Addr   => Jump_Addr);

```

process

begin

-- Test 1: NOP (opcode=000)

```

-- "000 000 000 000"
Instruction <= "000000000000"; wait for 20 ns;
assert Reg_En = "00000000" and Jump = '0' and Load_Imm = '0'
  report "FAIL: NOP defaults" severity error;

```

-- "000 000 000 000"

Instruction <= "000000000000"; wait for 20 ns;

assert Reg_En = "00000000" and Jump = '0' and Load_Imm = '0'

report "FAIL: NOP defaults" severity error;

-- Test 2: ADD R2, R0, R1

```

-- opcode=001, Rd=010, Rs1=000, Rs2=001
-- "001 010 000 001"
Instruction <= "001010000001"; wait for 20 ns;
assert Reg_En = "00000100" report "FAIL: ADD Reg_En (R2)"
  severity error;
assert Mux_Sel_A = "000" report "FAIL: ADD Mux_Sel_A=R0"
  severity error;
assert Mux_Sel_B = "001" report "FAIL: ADD Mux_Sel_B=R1"
  severity error;
assert Add_Sub = '0' report "FAIL: ADD Add_Sub=0"
  severity error;

```

-- opcode=001, Rd=010, Rs1=000, Rs2=001

-- "001 010 000 001"

Instruction <= "001010000001"; wait for 20 ns;

assert Reg_En = "00000100" report "FAIL: ADD Reg_En (R2)"

severity error;

assert Mux_Sel_A = "000" report "FAIL: ADD Mux_Sel_A=R0"

severity error;

assert Mux_Sel_B = "001" report "FAIL: ADD Mux_Sel_B=R1"

severity error;

assert Add_Sub = '0' report "FAIL: ADD Add_Sub=0"

severity error;


```

        assert Jump      = '0'                report "FAIL: MOVI Jump=0"
severity error;

-- Test 7: MOVI R1, #3
-- opcode=100, Rd=001, imm="0011"
-- "100 001 0011 00" â†’ "100001001100"
Instruction <= "100001001100"; wait for 20 ns;
assert Reg_En = "00000010"    report "FAIL: MOVI R1 Reg_En"
severity error;
assert Imm_Val = "0011"        report "FAIL: MOVI #3 Imm_Val"
severity error;
assert Load_Imm = '1'         report "FAIL: MOVI R1 Load_Imm"
severity error;

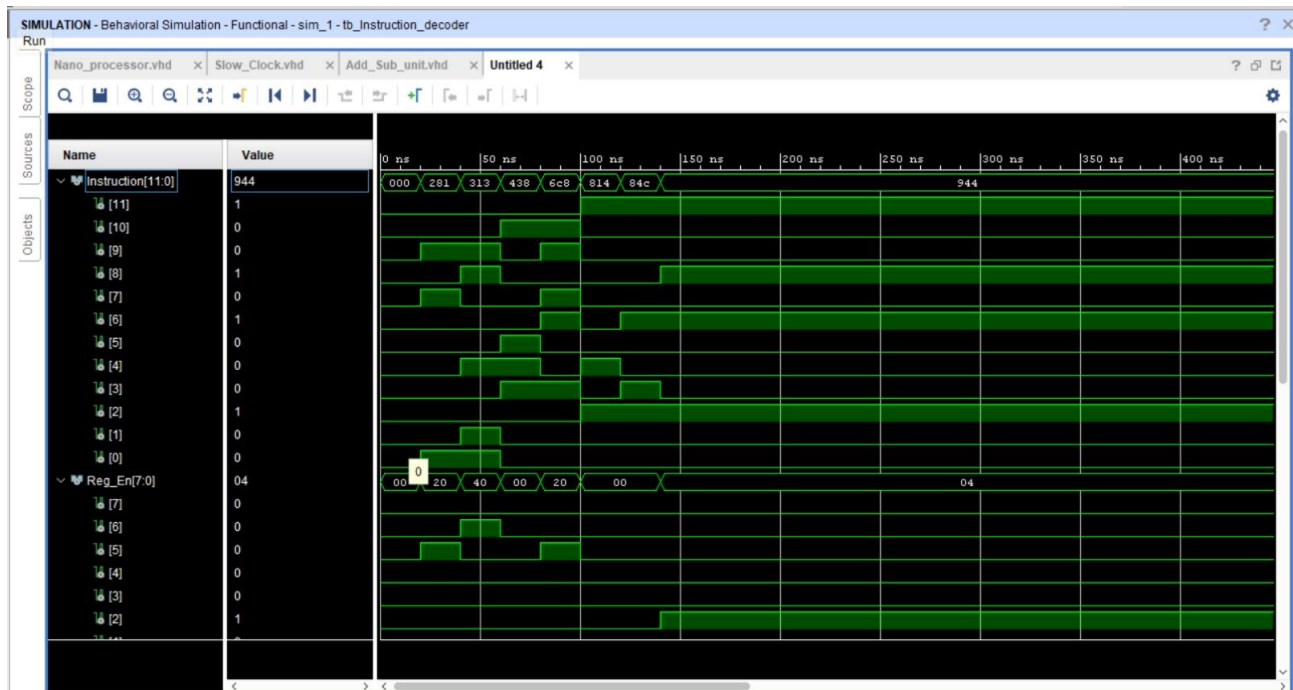
-- Test 8: MOVI R5, #1
-- opcode=100, Rd=101, imm="0001"
-- "100 101 0001 00" â†’ "100101000100"
Instruction <= "100101000100"; wait for 20 ns;
assert Reg_En = "00100000"    report "FAIL: MOVI R5 Reg_En"
severity error;
assert Imm_Val = "0001"        report "FAIL: MOVI #1 Imm_Val"
severity error;

        report "tb_Instruction_decoder: ALL TESTS DONE" severity note;
        wait;
    end process;

end Behavioral;

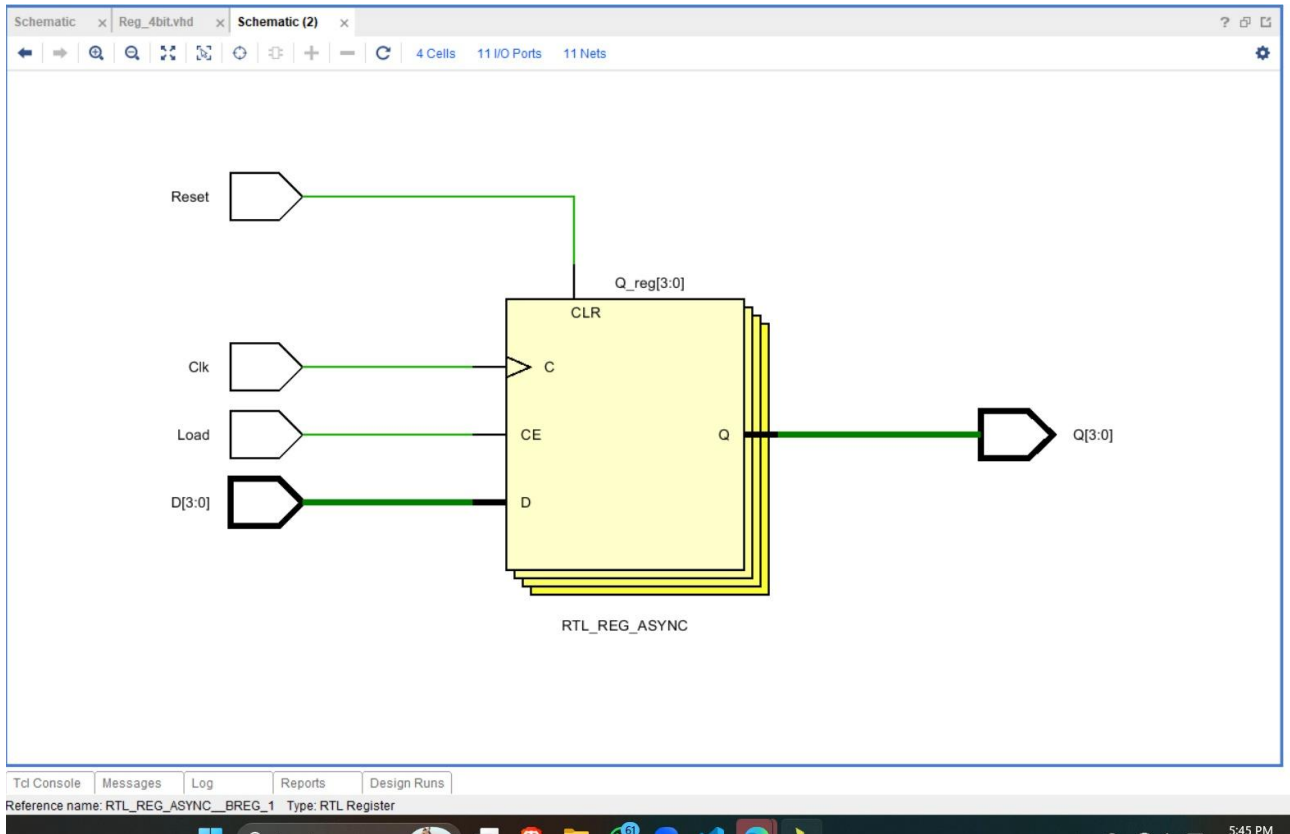
```

Timing Diagram



Reg_4bit.vhd

Schematic Diagram



Design Source Code

```
-----  
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:09:51 PM  
-- Design Name:  
-- Module Name: Reg_4bit - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--
```



```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Reg_4bit is
    Port ( D      : in  STD_LOGIC_VECTOR(3 downto 0);
          Load   : in  STD_LOGIC;
          Reset   : in  STD_LOGIC;
          Clk     : in  STD_LOGIC;
          Q       : out STD_LOGIC_VECTOR(3 downto 0));
end Reg_4bit;

architecture Behavioral of Reg_4bit is
begin
    process(Clk, Reset)
    begin
        if Reset = '1' then
            Q <= "0000";
        elsif rising_edge(Clk) then
            if Load = '1' then
                Q <= D;
            end if;
        end if;
    end process;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Reg_4bit
-- DUT       : Reg_4bit
-- Tests     : async reset, load on rising edge, hold when Load=0
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Reg_4bit is

```

```

end tb_Reg_4bit;

architecture Behavioral of tb_Reg_4bit is

    component Reg_4bit
        Port ( D      : in  STD_LOGIC_VECTOR(3 downto 0);
              Load   : in  STD_LOGIC;
              Reset   : in  STD_LOGIC;
              Clk     : in  STD_LOGIC;
              Q       : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal D      : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal Load   : STD_LOGIC := '0';
    signal Reset   : STD_LOGIC := '0';
    signal Clk    : STD_LOGIC := '0';
    signal Q      : STD_LOGIC_VECTOR(3 downto 0);

    constant CLK_PERIOD : time := 20 ns;

begin

    DUT: Reg_4bit port map(D=>D, Load=>Load, Reset=>Reset, Clk=>Clk, Q=>Q);

    Clk <= not Clk after CLK_PERIOD / 2;

    process
    begin
        -- Test 1: Async Reset â†’ Q = 0
        D <= "1111"; Load <= '1'; Reset <= '1'; wait for 15 ns;
        assert Q = "0000" report "FAIL: Reset" severity error;
        Reset <= '0'; wait for 5 ns;

        -- Test 2: Load 5 (0101)
        D <= "0101"; Load <= '1';
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "0101" report "FAIL: Load 5" severity error;

        -- Test 3: Load 10 (1010)
        D <= "1010";
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "1010" report "FAIL: Load 10" severity error;

        -- Test 4: Load = 0 â†’ holds 10
        D <= "0000"; Load <= '0';
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "1010" report "FAIL: Hold" severity error;

        -- Test 5: Load 15 (1111)
        D <= "1111"; Load <= '1';
        wait until rising_edge(Clk); wait for 5 ns;
        assert Q = "1111" report "FAIL: Load 15" severity error;

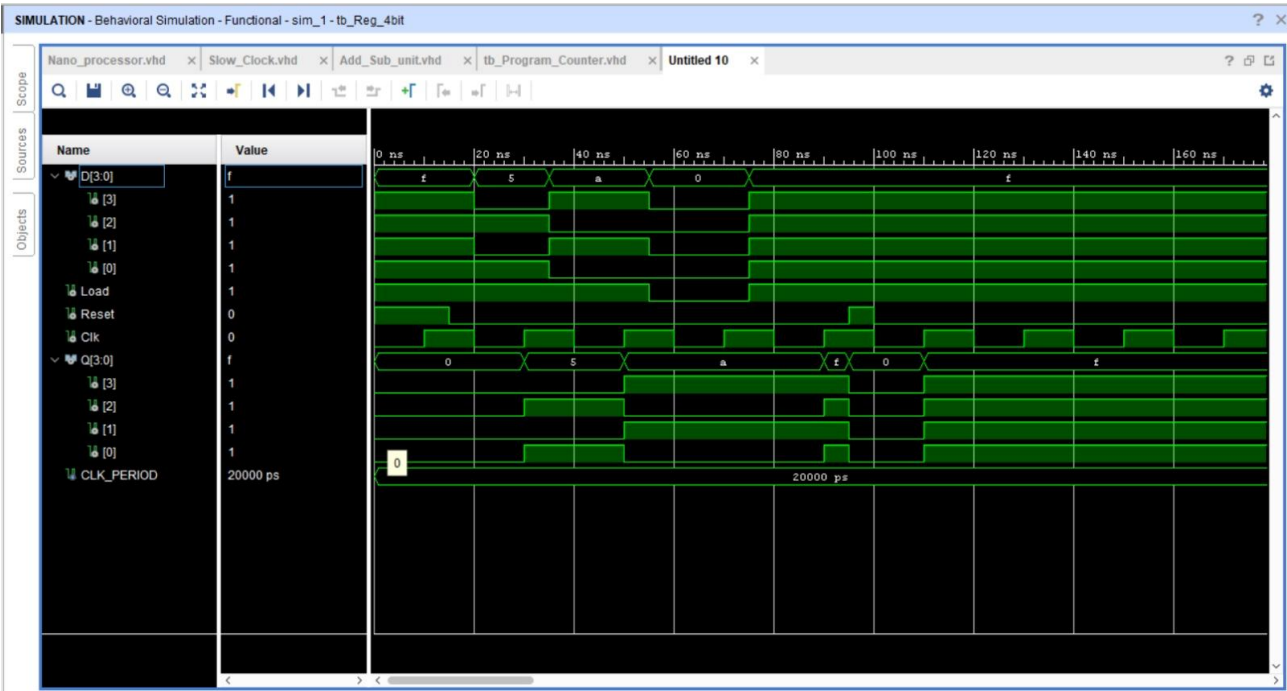
        -- Test 6: Reset mid-operation
        Reset <= '1'; wait for 5 ns;
        assert Q = "0000" report "FAIL: Mid-operation reset" severity error;
        Reset <= '0';

        report "tb_Reg_4bit: ALL TESTS DONE" severity note;
        wait;
    end process;

end Behavioral;

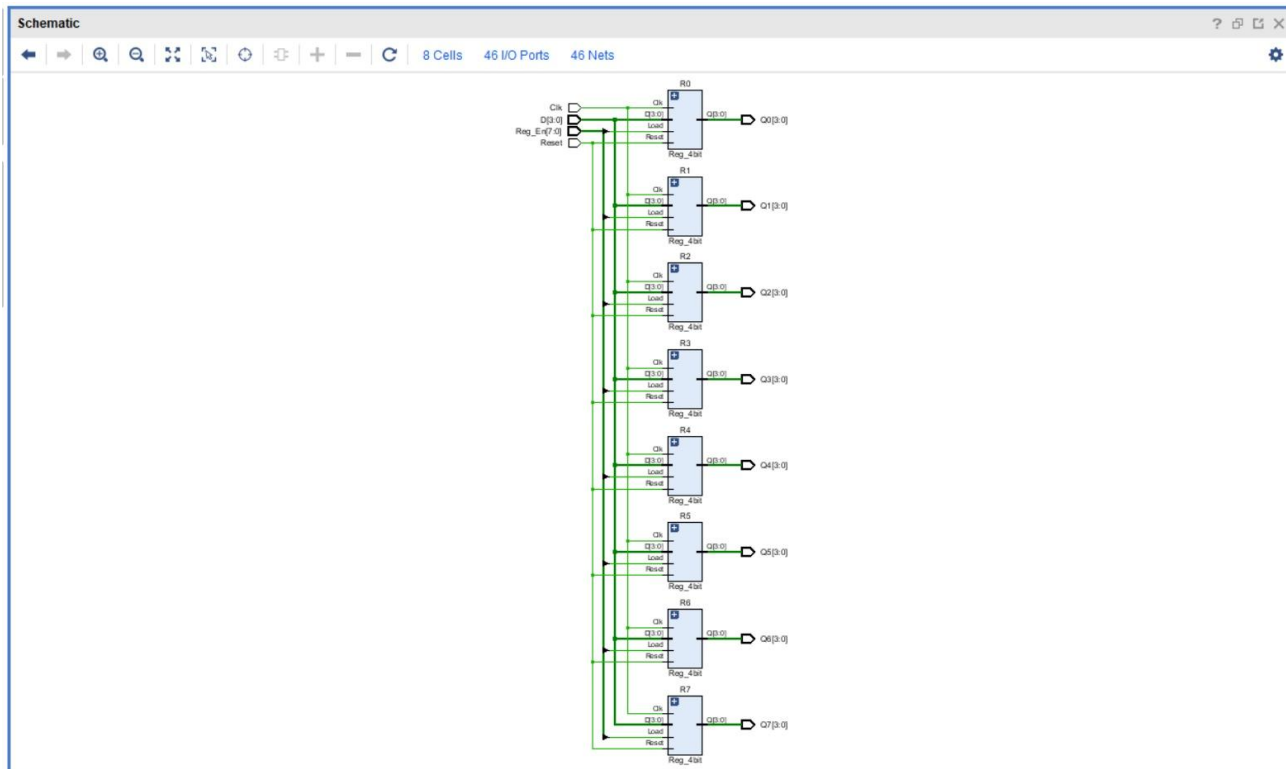
```

Timing Diagram



Registers_Bank.vhd

Schematic Diagram



Design Source Code

```
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:41:37 PM  
-- Design Name:  
-- Module Name: Registers_Bank - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Registers_Bank is
    Port ( D      : in  STD_LOGIC_VECTOR(3 downto 0);
          Reg_En  : in  STD_LOGIC_VECTOR(7 downto 0);
          Clk     : in  STD_LOGIC;
          Reset   : in  STD_LOGIC;
          Q0, Q1, Q2, Q3,
          Q4, Q5, Q6, Q7 : out STD_LOGIC_VECTOR(3 downto 0));
end Registers_Bank;

architecture Structural of Registers_Bank is
    component Reg_4bit
        Port ( D      : in  STD_LOGIC_VECTOR(3 downto 0);
              Load   : in  STD_LOGIC;
              Reset   : in  STD_LOGIC;
              Clk     : in  STD_LOGIC;
              Q       : out STD_LOGIC_VECTOR(3 downto 0));
    end component;
begin
    R0: Reg_4bit port map(D,Reg_En(0),Reset,Clk,Q0);
    R1: Reg_4bit port map(D,Reg_En(1),Reset,Clk,Q1);
    R2: Reg_4bit port map(D,Reg_En(2),Reset,Clk,Q2);
    R3: Reg_4bit port map(D,Reg_En(3),Reset,Clk,Q3);
    R4: Reg_4bit port map(D,Reg_En(4),Reset,Clk,Q4);
    R5: Reg_4bit port map(D,Reg_En(5),Reset,Clk,Q5);
    R6: Reg_4bit port map(D,Reg_En(6),Reset,Clk,Q6);
    R7: Reg_4bit port map(D,Reg_En(7),Reset,Clk,Q7);
end Structural;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Registers_Bank
-- DUT       : Registers_Bank (8 x Reg_4bit)
-- Tests     : write to each register via Reg_En, async reset, no-write hold
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Registers_Bank is
end tb_Registers_Bank;

architecture Behavioral of tb_Registers_Bank is

    component Registers_Bank
        Port ( D      : in  STD_LOGIC_VECTOR(3 downto 0);
              Reg_En  : in  STD_LOGIC_VECTOR(7 downto 0);
              Clk     : in  STD_LOGIC;
              Reset   : in  STD_LOGIC;
              Q0, Q1, Q2, Q3,
              Q4, Q5, Q6, Q7 : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

```

```

signal D      : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal Reg_En : STD_LOGIC_VECTOR(7 downto 0) := "00000000";
signal Clk     : STD_LOGIC := '0';
signal Reset   : STD_LOGIC := '0';
signal Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7 : STD_LOGIC_VECTOR(3 downto 0);

constant CLK_PERIOD : time := 20 ns;

begin

DUT: Registers_Bank port map(
    D=>D, Reg_En=>Reg_En, Clk=>Clk, Reset=>Reset,
    Q0=>Q0,Q1=>Q1,Q2=>Q2,Q3=>Q3,
    Q4=>Q4,Q5=>Q5,Q6=>Q6,Q7=>Q7);

Clk <= not Clk after CLK_PERIOD / 2;

process
begin
    -- Test 1: Async Reset â†’ all Q = 0
    Reset <= '1'; wait for 15 ns;
    assert Q0="0000" and Q1="0000" and Q2="0000" and Q3="0000" and
           Q4="0000" and Q5="0000" and Q6="0000" and Q7="0000"
        report "FAIL: Reset" severity error;
    Reset <= '0'; wait for 5 ns;

    -- Test 2: Write 5 to R1 (Reg_En bit 1)
    D <= "0101"; Reg_En <= "00000010";
    wait until rising_edge(Clk); wait for 5 ns;
    assert Q1 = "0101" report "FAIL: Write R1=5" severity error;
    assert Q0 = "0000" report "FAIL: R0 should stay 0 after R1 write"
severity error;

    -- Test 3: Write 3 to R2 (Reg_En bit 2)
    D <= "0011"; Reg_En <= "00000100";
    wait until rising_edge(Clk); wait for 5 ns;
    assert Q2 = "0011" report "FAIL: Write R2=3" severity error;
    assert Q1 = "0101" report "FAIL: R1 should hold 5" severity error;

    -- Test 4: Write 8 to R7 (Reg_En bit 7)
    D <= "1000"; Reg_En <= "10000000";
    wait until rising_edge(Clk); wait for 5 ns;
    assert Q7 = "1000" report "FAIL: Write R7=8" severity error;

    -- Test 5: No enable â†’ all registers hold values
    D <= "1111"; Reg_En <= "00000000";
    wait until rising_edge(Clk); wait for 5 ns;
    assert Q1 = "0101" report "FAIL: R1 hold" severity error;
    assert Q2 = "0011" report "FAIL: R2 hold" severity error;
    assert Q7 = "1000" report "FAIL: R7 hold" severity error;

    -- Test 6: Write 15 to R4
    D <= "1111"; Reg_En <= "00010000";
    wait until rising_edge(Clk); wait for 5 ns;
    assert Q4 = "1111" report "FAIL: Write R4=F" severity error;

    -- Test 7: Reset clears all
    Reset <= '1'; wait for 5 ns;
    assert Q1="0000" and Q2="0000" and Q4="0000" and Q7="0000"
        report "FAIL: Second reset" severity error;
    Reset <= '0';

    report "tb_Registers_Bank: ALL TESTS DONE" severity note;
end process;

```

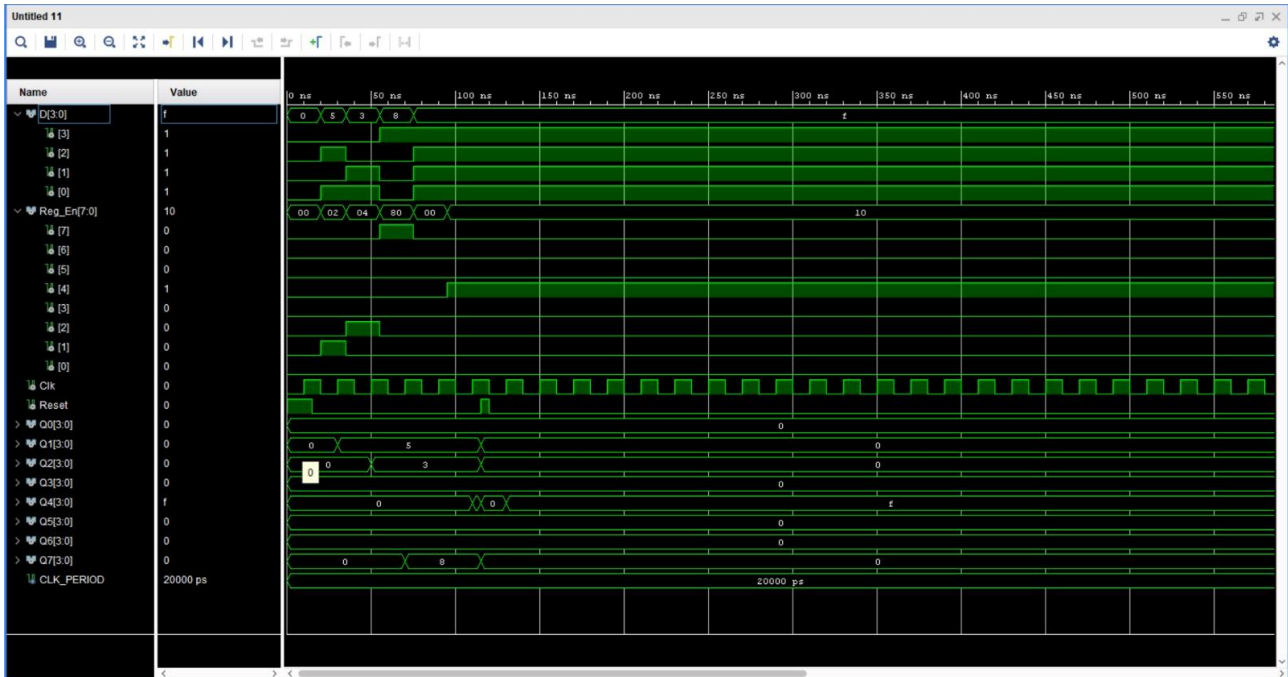
```

        wait;
    end process;

end Behavioral;

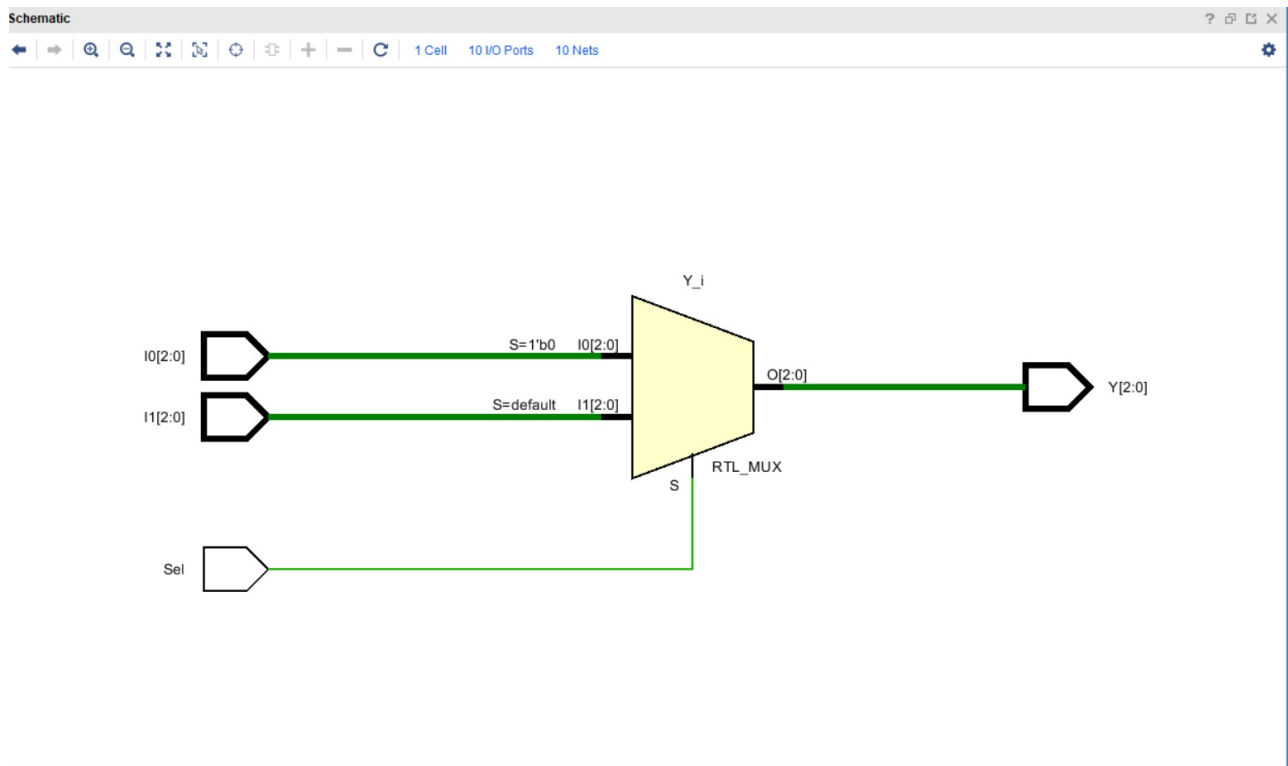
```

Timing Diagram



Mux_2_3bit.vhd

Schematic Diagram



Design Source Code

```
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:37:45 PM  
-- Design Name:  
-- Module Name: Mux_2_3bit - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
-- Uncomment the following library declaration if using
```



```

-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Mux_2_3bit is
    Port ( I0  : in  STD_LOGIC_VECTOR(2 downto 0);
          I1  : in  STD_LOGIC_VECTOR(2 downto 0);
          Sel  : in  STD_LOGIC;
          Y    : out STD_LOGIC_VECTOR(2 downto 0));
end Mux_2_3bit;

architecture Behavioral of Mux_2_3bit is
begin
    Y <= I0 when Sel = '0' else I1;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Mux_2_3bit
-- DUT       : Mux_2_3bit
-- Tests     : Sel=0 selects I0, Sel=1 selects I1, all combinations
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Mux_2_3bit is
end tb_Mux_2_3bit;

architecture Behavioral of tb_Mux_2_3bit is

    component Mux_2_3bit
        Port ( I0  : in  STD_LOGIC_VECTOR(2 downto 0);
              I1  : in  STD_LOGIC_VECTOR(2 downto 0);
              Sel  : in  STD_LOGIC;
              Y    : out STD_LOGIC_VECTOR(2 downto 0));
    end component;

    signal I0, I1 : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal Sel    : STD_LOGIC := '0';
    signal Y      : STD_LOGIC_VECTOR(2 downto 0);

begin

    DUT: Mux_2_3bit port map(I0=>I0, I1=>I1, Sel=>Sel, Y=>Y);

    process
    begin
        -- Test 1: Sel=0 â†’ Y = I0
        I0 <= "101"; I1 <= "010"; Sel <= '0'; wait for 20 ns;
        assert Y = "101" report "FAIL: Sel=0 should select I0" severity error;

        -- Test 2: Sel=1 â†’ Y = I1
        Sel <= '1'; wait for 20 ns;
    end process
end Behavioral;

```

```

assert Y = "010" report "FAIL: Sel=1 should select I1" severity error;

-- Test 3: Change I0, Sel=0
I0 <= "111"; Sel <= '0'; wait for 20 ns;
assert Y = "111" report "FAIL: I0=7, Sel=0" severity error;

-- Test 4: Change I1, Sel=1
I1 <= "110"; Sel <= '1'; wait for 20 ns;
assert Y = "110" report "FAIL: I1=6, Sel=1" severity error;

-- Test 5: Both same value
I0 <= "011"; I1 <= "011"; Sel <= '0'; wait for 20 ns;
assert Y = "011" report "FAIL: Both 3, Sel=0" severity error;

Sel <= '1'; wait for 20 ns;
assert Y = "011" report "FAIL: Both 3, Sel=1" severity error;

-- Test 6: PC use-case: I0=PC+1="100", I1=jump="001"
I0 <= "100"; I1 <= "001";
Sel <= '0'; wait for 20 ns;
assert Y = "100" report "FAIL: PC mux no-jump" severity error;
Sel <= '1'; wait for 20 ns;
assert Y = "001" report "FAIL: PC mux jump" severity error;

report "tb_Mux_2_3bit: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

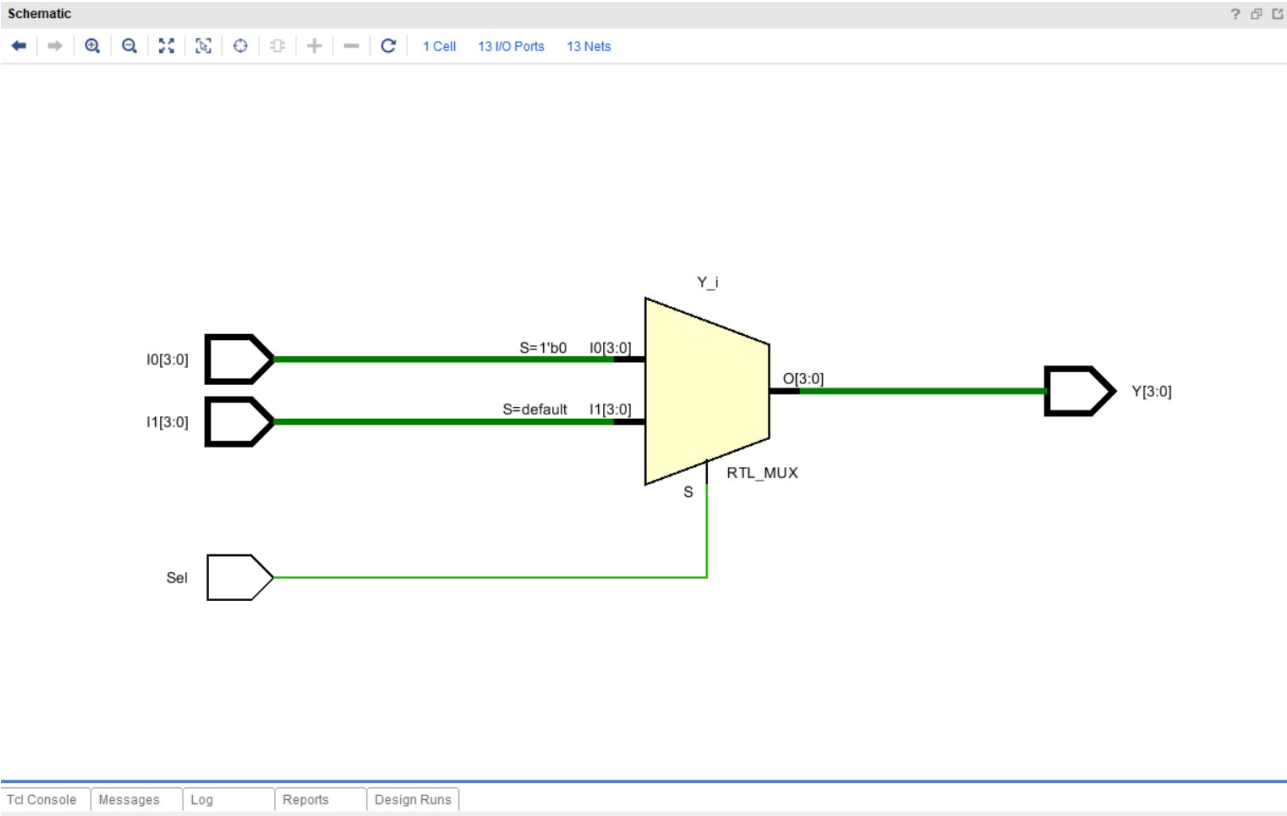
```

Timing Diagram



Mux 2 4bit.vhd

Schematic Diagram



Design Source Code

```
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:38:39 PM  
-- Design Name:  
-- Module Name: Mux_2_4bit - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--
```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Mux_2_4bit is
    Port ( I0   : in  STD_LOGIC_VECTOR(3 downto 0);
          I1   : in  STD_LOGIC_VECTOR(3 downto 0);
          Sel   : in  STD_LOGIC;
          Y     : out STD_LOGIC_VECTOR(3 downto 0));
end Mux_2_4bit;

architecture Behavioral of Mux_2_4bit is
begin
    Y <= I0 when Sel = '0' else I1;
end Behavioral;

```

Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Mux_2_4bit is
end tb_Mux_2_4bit;

architecture Behavioral of tb_Mux_2_4bit is

    component Mux_2_4bit
        Port ( I0   : in  STD_LOGIC_VECTOR(3 downto 0);
              I1   : in  STD_LOGIC_VECTOR(3 downto 0);
              Sel   : in  STD_LOGIC;
              Y     : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal I0, I1 : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal Sel    : STD_LOGIC := '0';
    signal Y      : STD_LOGIC_VECTOR(3 downto 0);

begin

    DUT: Mux_2_4bit port map(I0=>I0, I1=>I1, Sel=>Sel, Y=>Y);

    process
    begin
        -- Test 1: Sel=0 â†’ I0
        I0 <= "1010"; I1 <= "0101"; Sel <= '0'; wait for 20 ns;
        assert Y = "1010" report "FAIL: Sel=0, I0=A" severity error;

        -- Test 2: Sel=1 â†’ I1
        Sel <= '1'; wait for 20 ns;
        assert Y = "0101" report "FAIL: Sel=1, I1=5" severity error;
    end process
end Behavioral;

```

```

-- Test 3: write-back use-case: ALU=6, Imm=9, Sel=0 (ALU)
I0 <= "0110"; I1 <= "1001"; Sel <= '0'; wait for 20 ns;
assert Y = "0110" report "FAIL: WB mux ALU result" severity error;

-- Test 4: write-back use-case: Load_Imm=1 &+ ' I1=immediate
Sel <= '1'; wait for 20 ns;
assert Y = "1001" report "FAIL: WB mux immediate" severity error;

-- Test 5: All zeros
I0 <= "0000"; I1 <= "0000"; Sel <= '0'; wait for 20 ns;
assert Y = "0000" report "FAIL: Both 0, Sel=0" severity error;

-- Test 6: All ones
I0 <= "1111"; I1 <= "1111"; Sel <= '1'; wait for 20 ns;
assert Y = "1111" report "FAIL: Both F, Sel=1" severity error;

report "tb_Mux_2_4bit: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

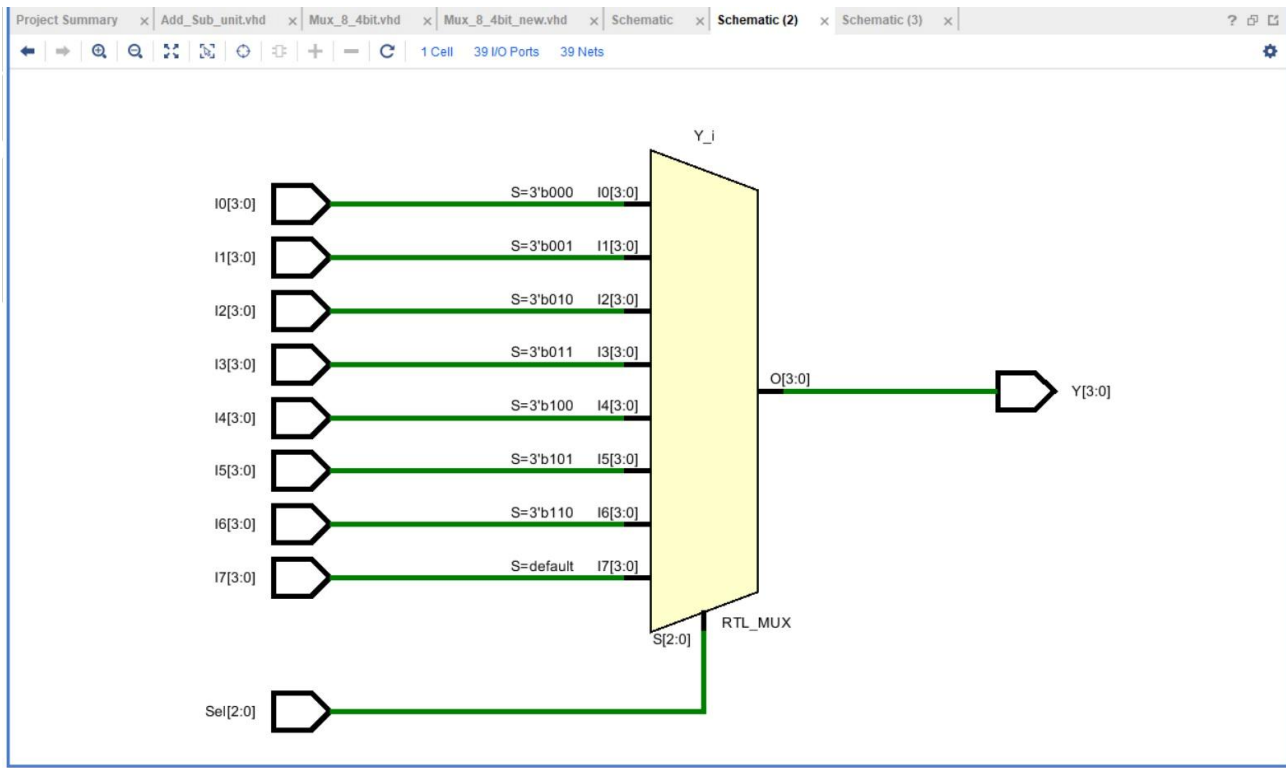
```

Timing Diagram



Mux 8 4bit.vhd

Schematic Diagram



Design Source Code

```
-----  
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:39:21 PM  
-- Design Name:  
-- Module Name: Mux_8_4bit - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Mux_8_4bit is
    Port ( I0, I1, I2, I3,
           I4, I5, I6, I7 : in  STD_LOGIC_VECTOR(3 downto 0);
           Sel              : in  STD_LOGIC_VECTOR(2 downto 0);
           Y                : out STD_LOGIC_VECTOR(3 downto 0));
end Mux_8_4bit;

architecture Behavioral of Mux_8_4bit is
begin
    with Sel select
        Y <= I0 when "000",
              I1 when "001",
              I2 when "010",
              I3 when "011",
              I4 when "100",
              I5 when "101",
              I6 when "110",
              I7 when others;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Mux_8_4bit
-- DUT       : Mux_8_4bit
-- Tests     : each of the 8 select lines picks the correct input
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Mux_8_4bit is
end tb_Mux_8_4bit;

architecture Behavioral of tb_Mux_8_4bit is

    component Mux_8_4bit
        Port ( I0, I1, I2, I3,
               I4, I5, I6, I7 : in  STD_LOGIC_VECTOR(3 downto 0);
               Sel              : in  STD_LOGIC_VECTOR(2 downto 0);
               Y                : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal I0 : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal I1 : STD_LOGIC_VECTOR(3 downto 0) := "0001";
    signal I2 : STD_LOGIC_VECTOR(3 downto 0) := "0010";
    signal I3 : STD_LOGIC_VECTOR(3 downto 0) := "0011";
    signal I4 : STD_LOGIC_VECTOR(3 downto 0) := "0100";
    signal I5 : STD_LOGIC_VECTOR(3 downto 0) := "0101";
    signal I6 : STD_LOGIC_VECTOR(3 downto 0) := "0110";

```

```

signal I7 : STD_LOGIC_VECTOR(3 downto 0) := "0111";
signal Sel : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Y   : STD_LOGIC_VECTOR(3 downto 0);

begin

DUT: Mux_8_4bit port map(
    I0=>I0,I1=>I1,I2=>I2,I3=>I3,
    I4=>I4,I5=>I5,I6=>I6,I7=>I7,
    Sel=>Sel, Y=>Y);

process
begin
    -- Each input is loaded with its index value for easy verification
    Sel <= "000"; wait for 20 ns;
    assert Y = "0000" report "FAIL: Sel=0" severity error;

    Sel <= "001"; wait for 20 ns;
    assert Y = "0001" report "FAIL: Sel=1" severity error;

    Sel <= "010"; wait for 20 ns;
    assert Y = "0010" report "FAIL: Sel=2" severity error;

    Sel <= "011"; wait for 20 ns;
    assert Y = "0011" report "FAIL: Sel=3" severity error;

    Sel <= "100"; wait for 20 ns;
    assert Y = "0100" report "FAIL: Sel=4" severity error;

    Sel <= "101"; wait for 20 ns;
    assert Y = "0101" report "FAIL: Sel=5" severity error;

    Sel <= "110"; wait for 20 ns;
    assert Y = "0110" report "FAIL: Sel=6" severity error;

    Sel <= "111"; wait for 20 ns;
    assert Y = "0111" report "FAIL: Sel=7" severity error;

    -- Test with non-trivial values on I3 and I5
    I3 <= "1011"; I5 <= "1101";
    Sel <= "011"; wait for 20 ns;
    assert Y = "1011" report "FAIL: I3 changed" severity error;

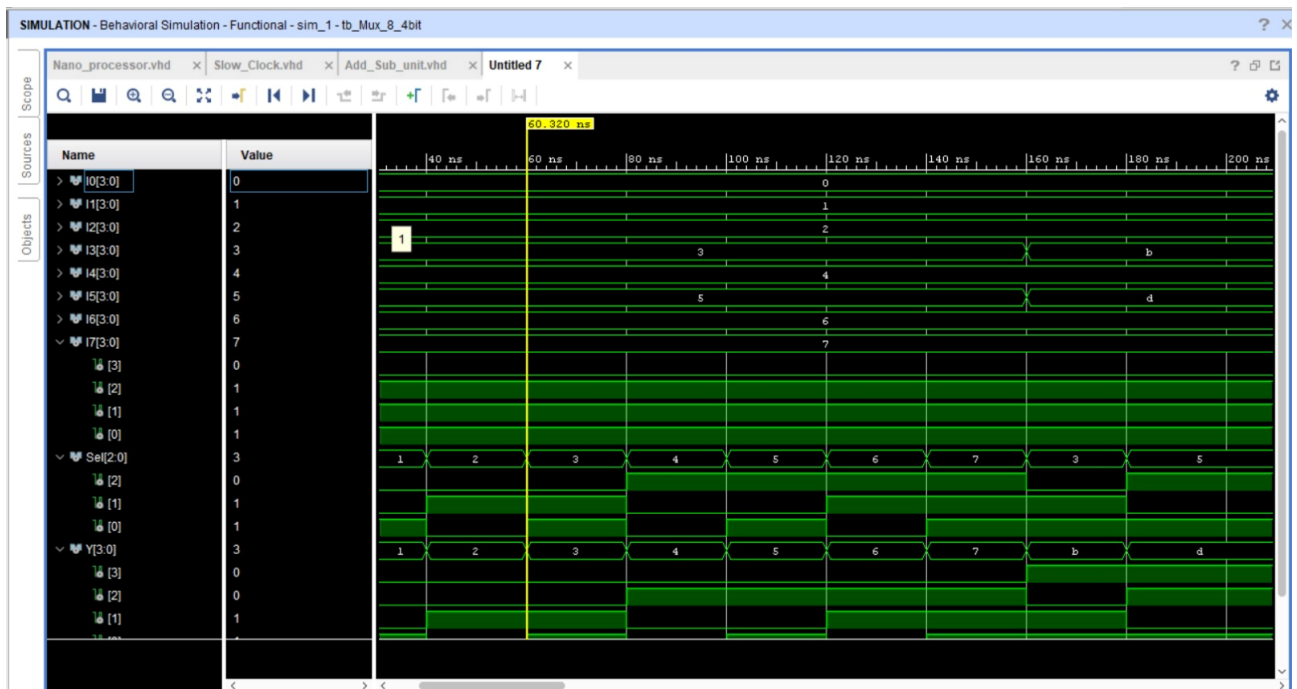
    Sel <= "101"; wait for 20 ns;
    assert Y = "1101" report "FAIL: I5 changed" severity error;

    report "tb_Mux_8_4bit: ALL TESTS DONE" severity note;
    wait;
end process;

end Behavioral;

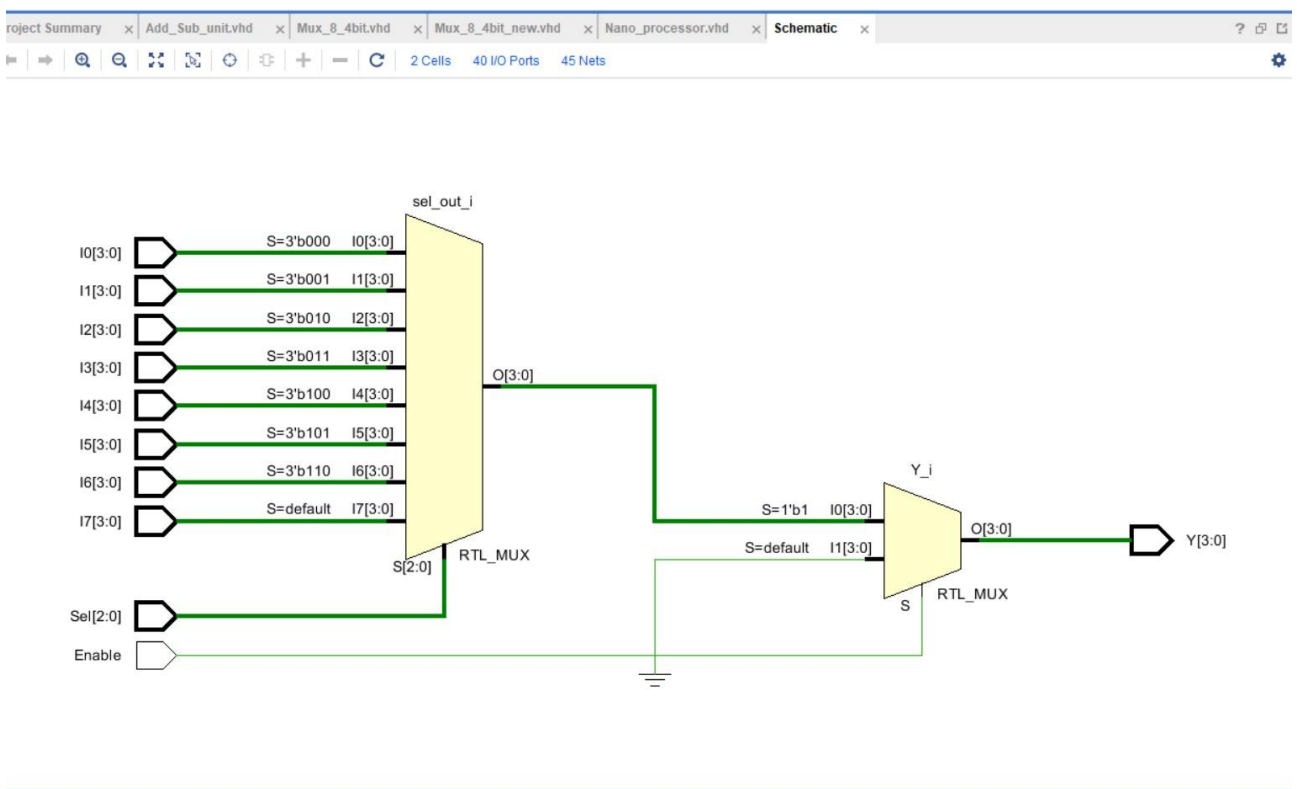
```


Timing Diagram



Mux 8 4bit new.vhd

Schematic Diagram



Design Source Code

```
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:40:02 PM
-- Design Name:
-- Module Name: Mux_8_4bit_new - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Mux_8_4bit_new is
    Port ( I0, I1, I2, I3,
           I4, I5, I6, I7 : in  STD_LOGIC_VECTOR(3 downto 0);
           Sel              : in  STD_LOGIC_VECTOR(2 downto 0);
           Enable           : in  STD_LOGIC;
           Y                : out STD_LOGIC_VECTOR(3 downto 0));
end Mux_8_4bit_new;

architecture Behavioral of Mux_8_4bit_new is
    signal sel_out : STD_LOGIC_VECTOR(3 downto 0);
begin
    with Sel select
        sel_out <= I0 when "000",
                   I1 when "001",
                   I2 when "010",
                   I3 when "011",
                   I4 when "100",
                   I5 when "101",
                   I6 when "110",
                   I7 when others;
    Y <= sel_out when Enable = '1' else "0000";
end Behavioral;
```

Simulation Source Code

```
-----
--
-- Testbench : tb_Mux_8_4bit_new
-- DUT       : Mux_8_4bit_new
-- Tests     : selection with Enable=1 and output=0000 when Enable=0
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Mux_8_4bit_new is
end tb_Mux_8_4bit_new;

architecture Behavioral of tb_Mux_8_4bit_new is

    component Mux_8_4bit_new
        Port ( I0, I1, I2, I3,
              I4, I5, I6, I7 : in  STD_LOGIC_VECTOR(3 downto 0);
              Sel            : in  STD_LOGIC_VECTOR(2 downto 0);
              Enable         : in  STD_LOGIC;
              Y              : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal I0 : STD_LOGIC_VECTOR(3 downto 0) := "1000";
    signal I1 : STD_LOGIC_VECTOR(3 downto 0) := "1001";
    signal I2 : STD_LOGIC_VECTOR(3 downto 0) := "1010";
    signal I3 : STD_LOGIC_VECTOR(3 downto 0) := "1011";
    signal I4 : STD_LOGIC_VECTOR(3 downto 0) := "1100";
    signal I5 : STD_LOGIC_VECTOR(3 downto 0) := "1101";
    signal I6 : STD_LOGIC_VECTOR(3 downto 0) := "1110";
    signal I7 : STD_LOGIC_VECTOR(3 downto 0) := "1111";
    signal Sel : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal Enable : STD_LOGIC := '0';
    signal Y : STD_LOGIC_VECTOR(3 downto 0);

begin

    DUT: Mux_8_4bit_new port map(
        I0=>I0,I1=>I1,I2=>I2,I3=>I3,
        I4=>I4,I5=>I5,I6=>I6,I7=>I7,
        Sel=>Sel, Enable=>Enable, Y=>Y);

    process
    begin
        -- Enable = 0 -> Y must be 0000 regardless of Sel
        Enable <= '0';
        Sel <= "000"; wait for 20 ns;
        assert Y = "0000" report "FAIL: Enable=0, Sel=0" severity error;

        Sel <= "101"; wait for 20 ns;
        assert Y = "0000" report "FAIL: Enable=0, Sel=5" severity error;

        -- Enable = 1 -> normal mux selection
        Enable <= '1';
        Sel <= "000"; wait for 20 ns;
        assert Y = "1000" report "FAIL: Enable=1, Sel=0" severity error;

        Sel <= "001"; wait for 20 ns;
    end process
end;
```

```

assert Y = "1001" report "FAIL: Enable=1, Sel=1" severity error;

Sel <= "010"; wait for 20 ns;
assert Y = "1010" report "FAIL: Enable=1, Sel=2" severity error;

Sel <= "011"; wait for 20 ns;
assert Y = "1011" report "FAIL: Enable=1, Sel=3" severity error;

Sel <= "100"; wait for 20 ns;
assert Y = "1100" report "FAIL: Enable=1, Sel=4" severity error;

Sel <= "101"; wait for 20 ns;
assert Y = "1101" report "FAIL: Enable=1, Sel=5" severity error;

Sel <= "110"; wait for 20 ns;
assert Y = "1110" report "FAIL: Enable=1, Sel=6" severity error;

Sel <= "111"; wait for 20 ns;
assert Y = "1111" report "FAIL: Enable=1, Sel=7" severity error;

-- Toggle Enable mid-operation
Sel <= "011"; Enable <= '1'; wait for 20 ns;
assert Y = "1011" report "FAIL: Toggle on, Sel=3" severity error;

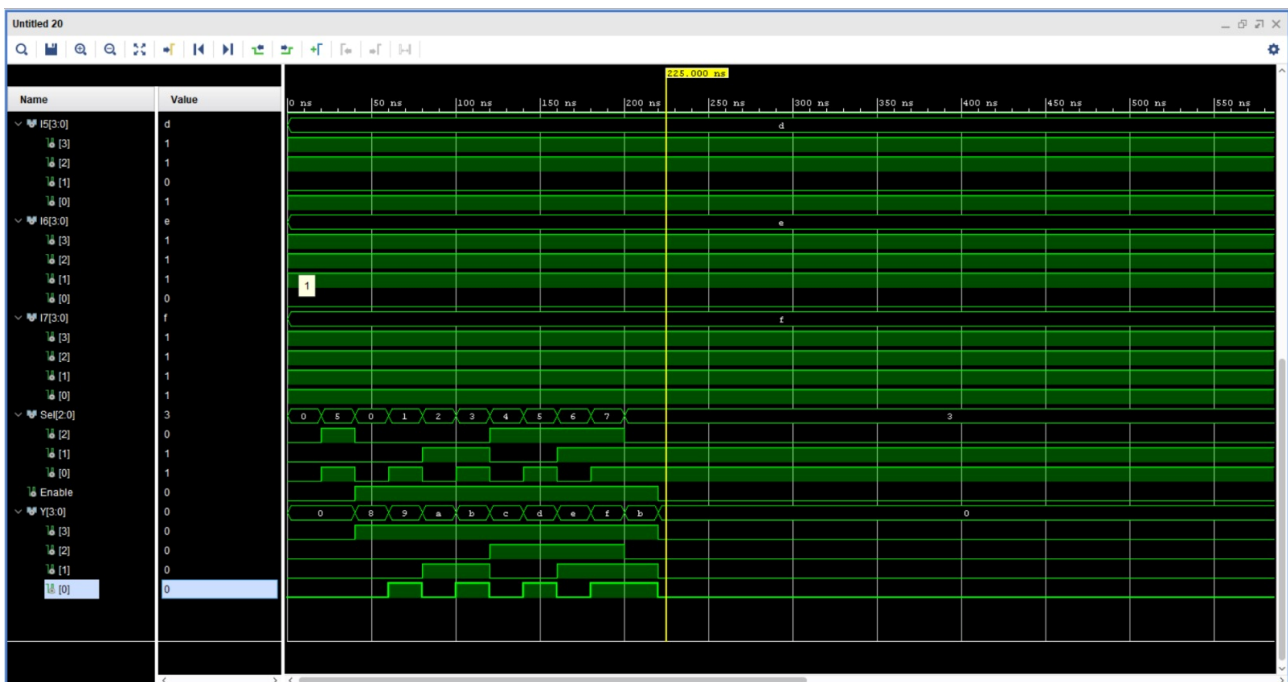
Enable <= '0'; wait for 20 ns;
assert Y = "0000" report "FAIL: Toggle off" severity error;

report "tb_Mux_8_4bit_new: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

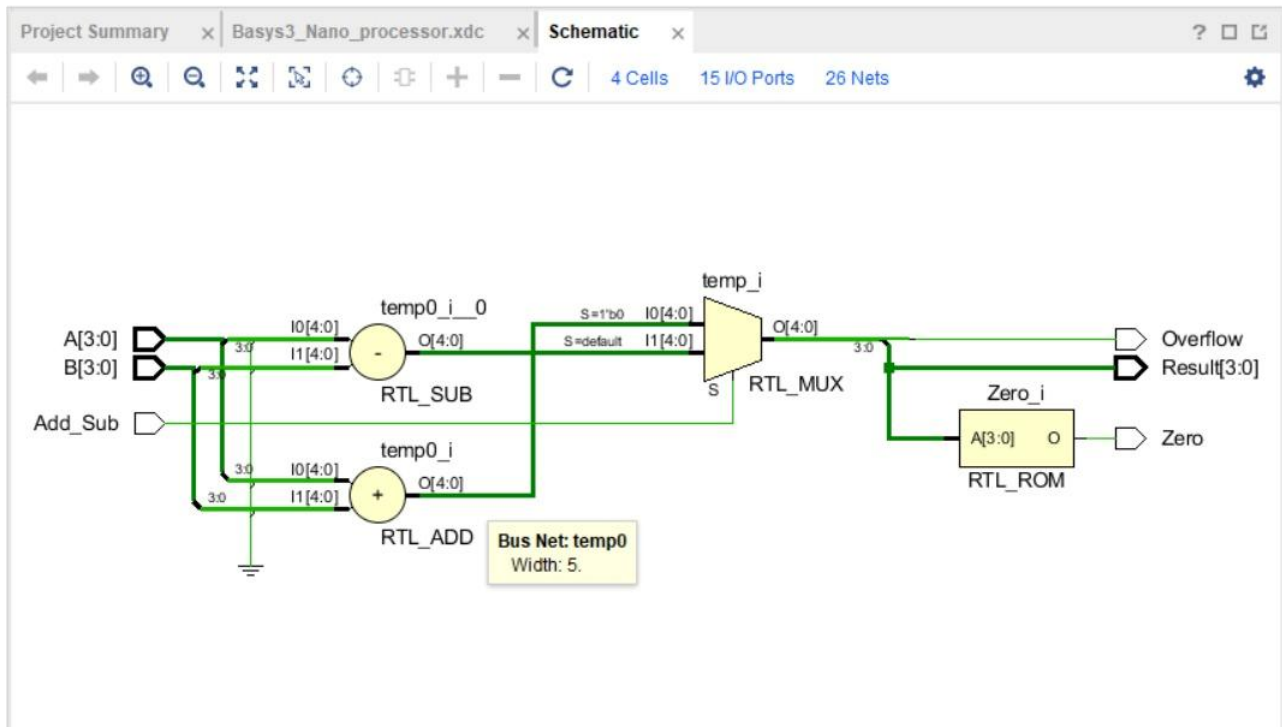
```

Timing Diagram



Add_Sub_unit.vhd

Schematic Diagram



Design Source Code

```
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:36:56 PM
-- Design Name:
-- Module Name: Add_Sub_unit - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
```

```
-- Dependencies:
```

```
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
```

```

--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

use IEEE.NUMERIC_STD.ALL;

entity Add_Sub_unit is
    Port ( A      : in  STD_LOGIC_VECTOR(3 downto 0);
          B      : in  STD_LOGIC_VECTOR(3 downto 0);
          Add_Sub : in  STD_LOGIC; -- 0=Add, 1=Sub
          Result  : out STD_LOGIC_VECTOR(3 downto 0);
          Overflow : out STD_LOGIC;
          Zero     : out STD_LOGIC);
end Add_Sub_unit;

architecture Behavioral of Add_Sub_unit is
    signal temp : STD_LOGIC_VECTOR(4 downto 0);
begin
    process(A, B, Add_Sub)
    begin
        if Add_Sub = '0' then
            temp <= std_logic_vector(
                unsigned('0' & A) + unsigned('0' & B));
        else
            temp <= std_logic_vector(
                unsigned('0' & A) - unsigned('0' & B));
        end if;
    end process;

    Result  <= temp(3 downto 0);
    Overflow <= temp(4);
    Zero    <= '1' when temp(3 downto 0) = "0000" else '0';
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Add_Sub_unit
-- DUT       : Add_Sub_unit
-- Tests     : addition, subtraction, overflow, zero flag
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_Add_Sub_unit is
end tb_Add_Sub_unit;

architecture Behavioral of tb_Add_Sub_unit is

    component Add_Sub_unit
        Port ( A      : in  STD_LOGIC_VECTOR(3 downto 0);
              B      : in  STD_LOGIC_VECTOR(3 downto 0);
              Add_Sub : in  STD_LOGIC;
              Result  : out STD_LOGIC_VECTOR(3 downto 0);
              Overflow : out STD_LOGIC;
              Zero     : out STD_LOGIC);
    end component;

```



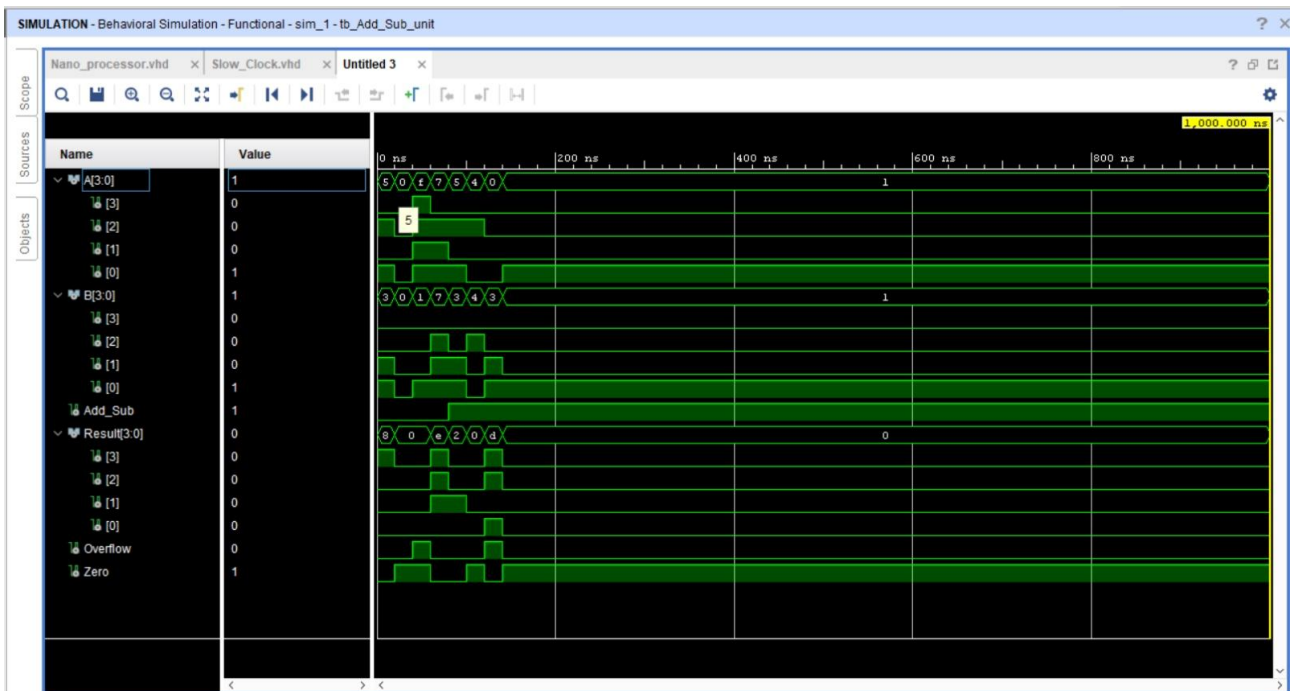
```

end process;

end Behavioral;

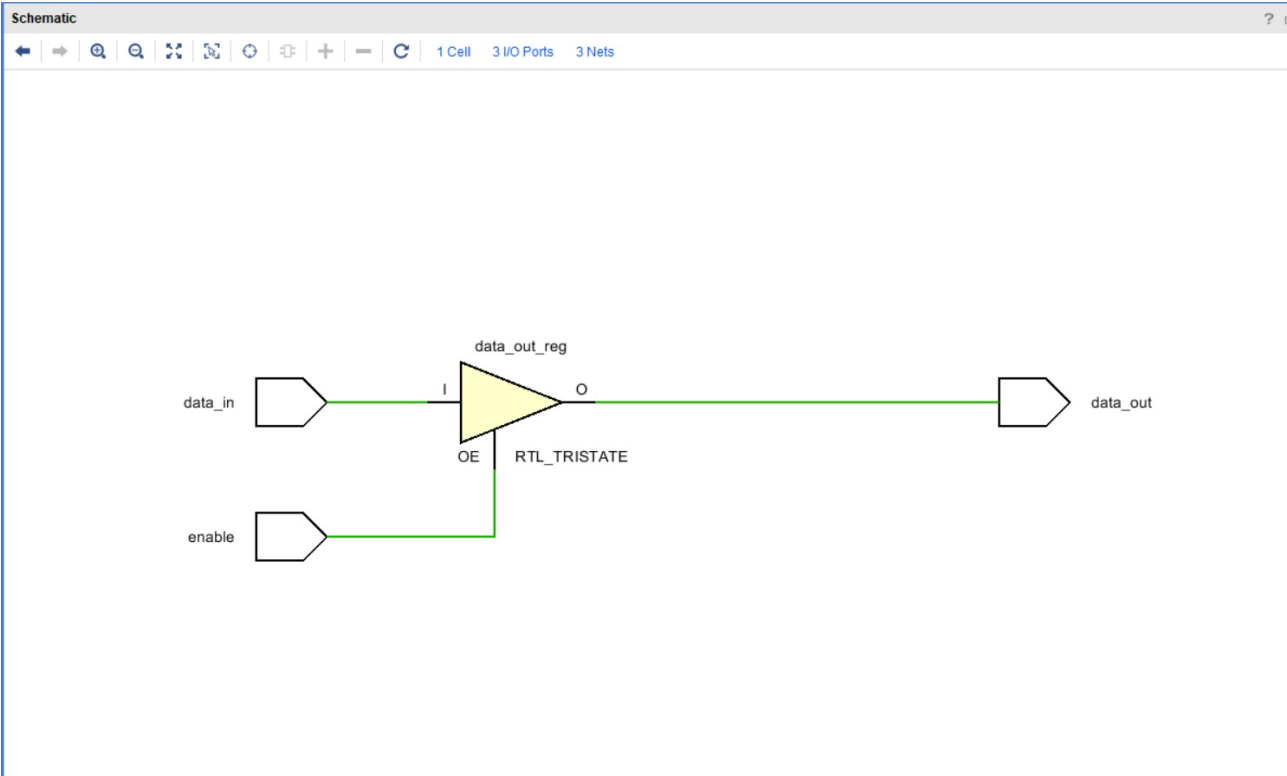
```

Timing Diagram



Tri State buffer.vhd

Schematic Diagram



Design Source Code

```
--
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 01:59:44 PM
-- Design Name:
-- Module Name: Tri_State_buffer - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Tri_State_buffer is
    Port ( data_in  : in  STD_LOGIC;
          enable   : in  STD_LOGIC;
          data_out  : out STD_LOGIC);
end Tri_State_buffer;

architecture Behavioral of Tri_State_buffer is
begin
    data_out <= data_in when enable = '1' else 'Z';
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Tri_State_buffer
-- DUT       : Tri_State_buffer (1-bit)
-- Tests     : enable=1 passes data, enable=0 outputs 'Z'
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Tri_State_buffer is
end tb_Tri_State_buffer;

architecture Behavioral of tb_Tri_State_buffer is

    component Tri_State_buffer
        Port ( data_in  : in  STD_LOGIC;
              enable   : in  STD_LOGIC;
              data_out  : out STD_LOGIC);
    end component;

    signal data_in   : STD_LOGIC := '0';
    signal enable    : STD_LOGIC := '0';
    signal data_out  : STD_LOGIC;

begin

    DUT: Tri_State_buffer port map(data_in=>data_in, enable=>enable,
data_out=>data_out);

    process
    begin
        -- Test 1: enable=1, data_in=0 â†’ data_out=0
        data_in <= '0'; enable <= '1'; wait for 20 ns;
        assert data_out = '0' report "FAIL: en=1, in=0" severity error;

        -- Test 2: enable=1, data_in=1 â†’ data_out=1

```

```

data_in <= '1'; enable <= '1'; wait for 20 ns;
assert data_out = '1' report "FAIL: en=1, in=1" severity error;

-- Test 3: enable=0, data_in=1 → data_out=Z
data_in <= '1'; enable <= '0'; wait for 20 ns;
assert data_out = 'Z' report "FAIL: en=0, should be Z" severity error;

-- Test 4: enable=0, data_in=0 → data_out=Z
data_in <= '0'; enable <= '0'; wait for 20 ns;
assert data_out = 'Z' report "FAIL: en=0, in=0, should be Z" severity
error;

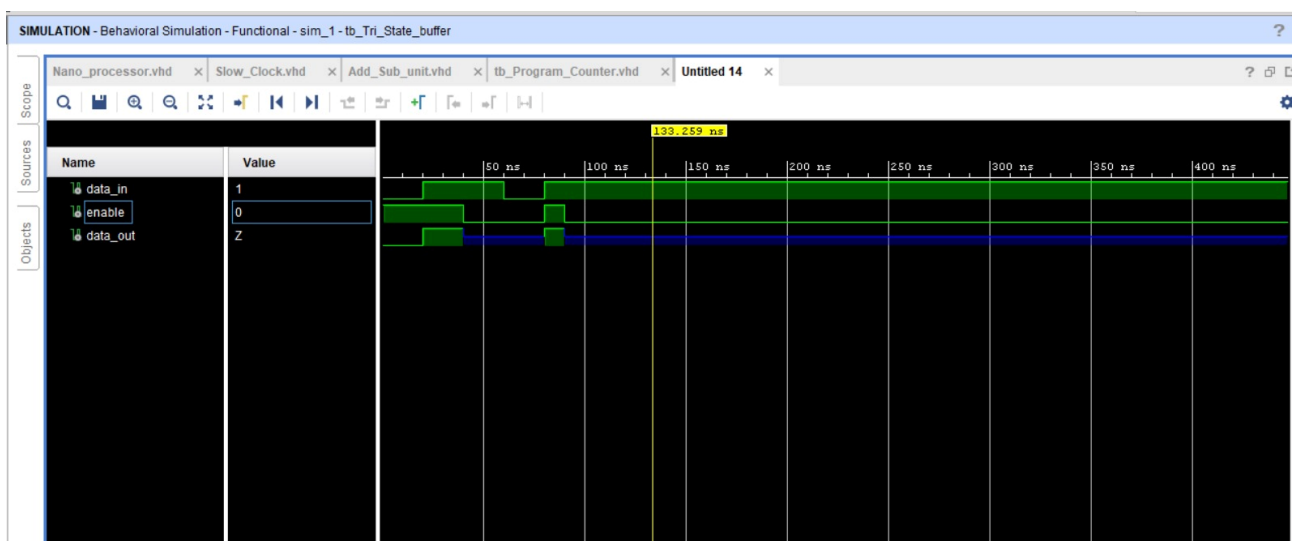
-- Test 5: Toggle enable rapidly
data_in <= '1'; enable <= '1'; wait for 10 ns;
assert data_out = '1' report "FAIL: rapid toggle on" severity error;
enable <= '0'; wait for 10 ns;
assert data_out = 'Z' report "FAIL: rapid toggle off" severity error;

report "tb_Tri_State_buffer: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

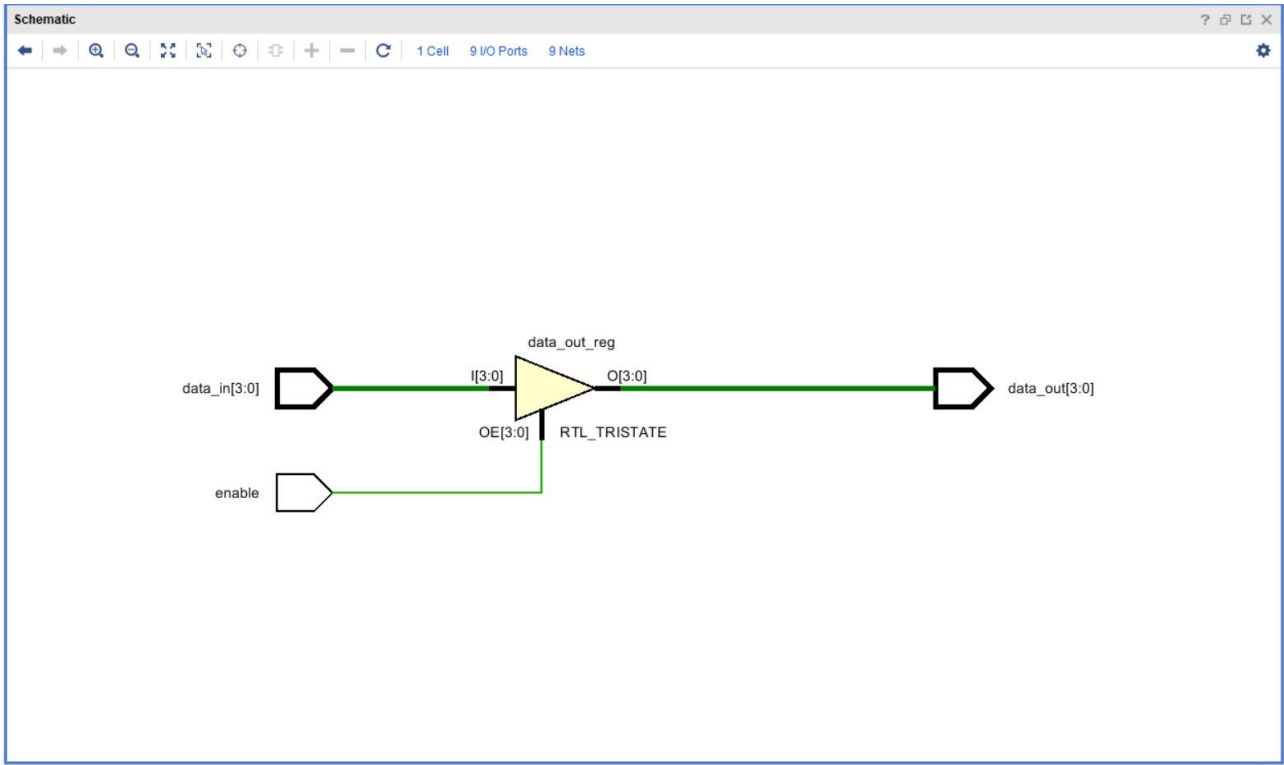
```

Timing Diagram



Tri State buffer 4bit.vhd

Schematic Diagram



Design Source Code

```
-----  
--  
-- Company:  
-- Engineer:  
--  
-- Create Date: 04/29/2026 02:00:59 PM  
-- Design Name:  
-- Module Name: Tri_State_buffer_4bit - Behavioral  
-- Project Name:  
-- Target Devices:  
-- Tool Versions:  
-- Description:  
--  
-- Dependencies:  
--  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
--  
  
library IEEE;
```

```

use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Tri_State_buffer_4bit is
    Port ( data_in  : in  STD_LOGIC_VECTOR(3 downto 0);
          enable   : in  STD_LOGIC;
          data_out  : out STD_LOGIC_VECTOR(3 downto 0));
end Tri_State_buffer_4bit;

architecture Behavioral of Tri_State_buffer_4bit is
begin
    data_out <= data_in when enable = '1' else "ZZZZ";
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Tri_State_buffer_4bit
-- DUT       : Tri_State_buffer_4bit
-- Tests     : enable=1 passes bus, enable=0 outputs "ZZZZ"
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Tri_State_buffer_4bit is
end tb_Tri_State_buffer_4bit;

architecture Behavioral of tb_Tri_State_buffer_4bit is

    component Tri_State_buffer_4bit
        Port ( data_in  : in  STD_LOGIC_VECTOR(3 downto 0);
              enable   : in  STD_LOGIC;
              data_out  : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal data_in  : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal enable   : STD_LOGIC := '0';
    signal data_out : STD_LOGIC_VECTOR(3 downto 0);

begin

    DUT: Tri_State_buffer_4bit port map(
        data_in=>data_in, enable=>enable, data_out=>data_out);

    process
    begin
        -- Test 1: enable=1, data=5 â†’ output=5
        data_in <= "0101"; enable <= '1'; wait for 20 ns;
        assert data_out = "0101" report "FAIL: en=1, data=5" severity error;

        -- Test 2: enable=1, data=A â†’ output=A
        data_in <= "1010"; enable <= '1'; wait for 20 ns;
        assert data_out = "1010" report "FAIL: en=1, data=A" severity error;
    end process
end architecture Behavioral;

```

```

-- Test 3: enable=1, data=F
data_in <= "1111"; enable <= '1'; wait for 20 ns;
assert data_out = "1111" report "FAIL: en=1, data=F" severity error;

-- Test 4: enable=0 â†’ high-impedance
data_in <= "1111"; enable <= '0'; wait for 20 ns;
assert data_out = "ZZZZ" report "FAIL: en=0, should be ZZZZ" severity
error;

-- Test 5: enable=0, data=0 â†’ still Z
data_in <= "0000"; enable <= '0'; wait for 20 ns;
assert data_out = "ZZZZ" report "FAIL: en=0, data=0, should be ZZZZ"
severity error;

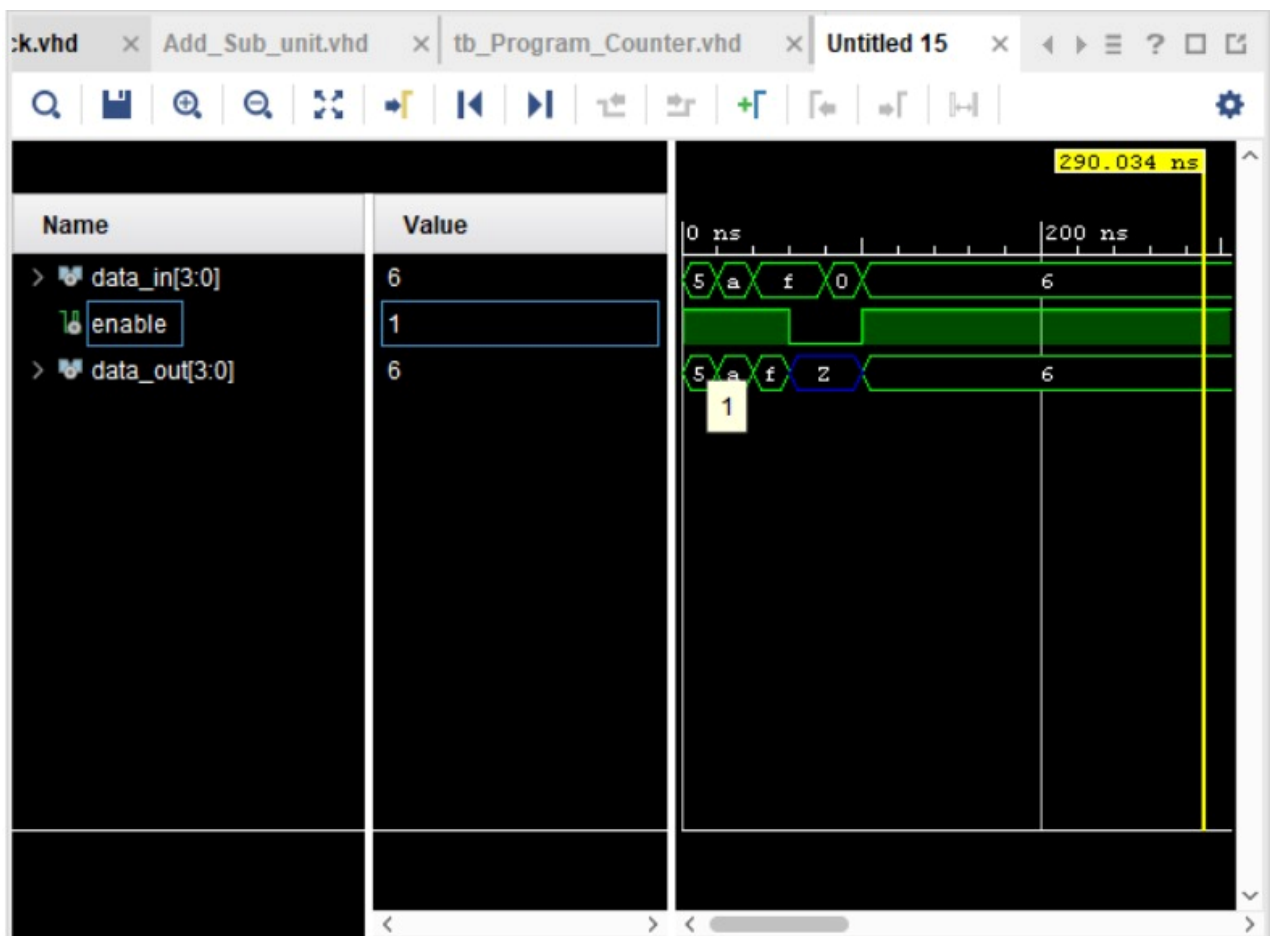
-- Test 6: Re-enable â†’ output passes through
data_in <= "0110"; enable <= '1'; wait for 20 ns;
assert data_out = "0110" report "FAIL: re-enable" severity error;

report "tb_Tri_State_buffer_4bit: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

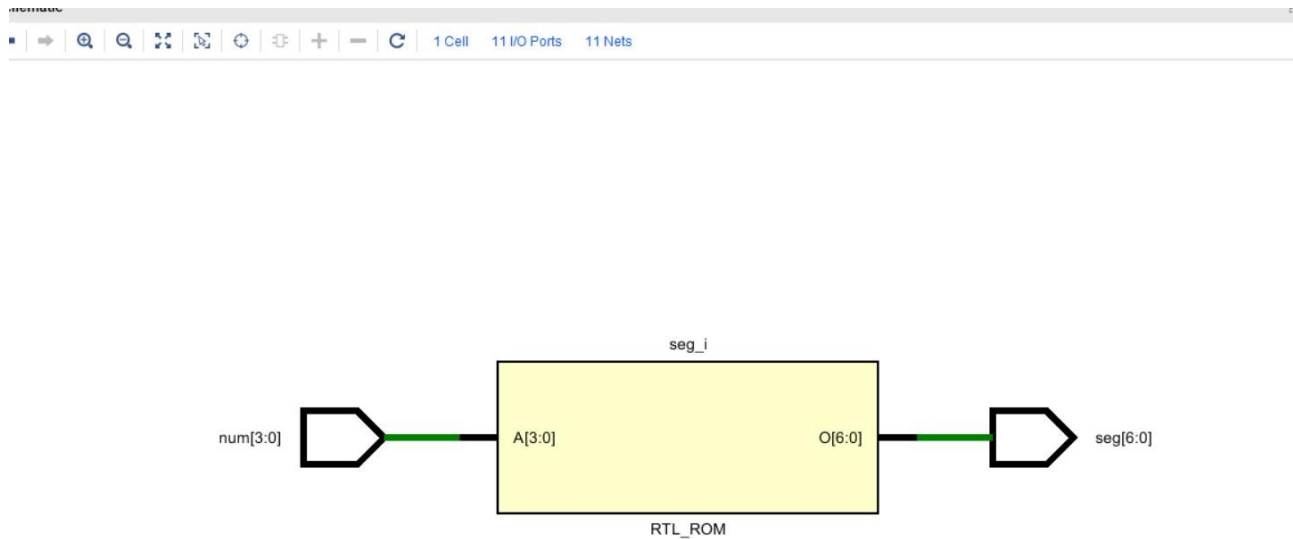
```

Timing Diagram



Seven_Seg_Decoder.vhd

Schematic Diagram



Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- =====
-- Seven_Seg_Decoder
-- Converts 4-bit binary value to 7-segment display encoding
-- Active LOW cathodes (Basys3 standard)
-- seg(6 downto 0) = gfedcba
--   seg(0)=a (top), seg(1)=b (top-right), seg(2)=c (bot-right)
--   seg(3)=d (bot),  seg(4)=e (bot-left),  seg(5)=f (top-left)
--   seg(6)=g (middle)
-- Displays 0â€"9 as decimal, Aâ€"F as hex
-- =====
entity Seven_Seg_Decoder is
    Port ( num : in  STD_LOGIC_VECTOR(3 downto 0);
          seg : out STD_LOGIC_VECTOR(6 downto 0)); -- seg(6)=g ... seg(0)=a
end Seven_Seg_Decoder;

architecture Behavioral of Seven_Seg_Decoder is
begin
    process(num)
    begin
        case num is
            when "0000" => seg <= "1000000"; -- 0
            when "0001" => seg <= "1111001"; -- 1
            when "0010" => seg <= "0100100"; -- 2
```

```

        when "0011" => seg <= "0110000"; -- 3
        when "0100" => seg <= "0011001"; -- 4
        when "0101" => seg <= "0010010"; -- 5
        when "0110" => seg <= "0000010"; -- 6
        when "0111" => seg <= "1111000"; -- 7
        when "1000" => seg <= "0000000"; -- 8
        when "1001" => seg <= "0010000"; -- 9
        when others => seg <= "1111111"; -- blank
    end case;
end process;
end Behavioral;

```

Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Seven_Seg_Decoder is
end tb_Seven_Seg_Decoder;

architecture Behavioral of tb_Seven_Seg_Decoder is

    component Seven_Seg_Decoder
        Port ( num : in  STD_LOGIC_VECTOR(3 downto 0);
              seg : out STD_LOGIC_VECTOR(6 downto 0));
    end component;

    signal num : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    signal seg : STD_LOGIC_VECTOR(6 downto 0);

begin

    DUT: Seven_Seg_Decoder port map(num=>num, seg=>seg);

    process
    begin
        num <= "0000"; wait for 20 ns; -- 0
        assert seg = "1000000" report "FAIL: digit 0" severity error;

        num <= "0001"; wait for 20 ns; -- 1
        assert seg = "1111001" report "FAIL: digit 1" severity error;

        num <= "0010"; wait for 20 ns; -- 2
        assert seg = "0100100" report "FAIL: digit 2" severity error;

        num <= "0011"; wait for 20 ns; -- 3
        assert seg = "0110000" report "FAIL: digit 3" severity error;

        num <= "0100"; wait for 20 ns; -- 4
        assert seg = "0011001" report "FAIL: digit 4" severity error;

        num <= "0101"; wait for 20 ns; -- 5
        assert seg = "0010010" report "FAIL: digit 5" severity error;

        num <= "0110"; wait for 20 ns; -- 6
        assert seg = "0000010" report "FAIL: digit 6" severity error;

        num <= "0111"; wait for 20 ns; -- 7
        assert seg = "1111000" report "FAIL: digit 7" severity error;

        num <= "1000"; wait for 20 ns; -- 8
    end process;
end Behavioral;

```



```

assert seg = "0000000" report "FAIL: digit 8" severity error;

num <= "1001"; wait for 20 ns; -- 9
assert seg = "0010000" report "FAIL: digit 9" severity error;

-- Values 10-15 should produce "1111111" (blank) per your decoder
num <= "1010"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit A (blank)" severity error;

num <= "1011"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit B (blank)" severity error;

num <= "1100"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit C (blank)" severity error;

num <= "1101"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit D (blank)" severity error;

num <= "1110"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit E (blank)" severity error;

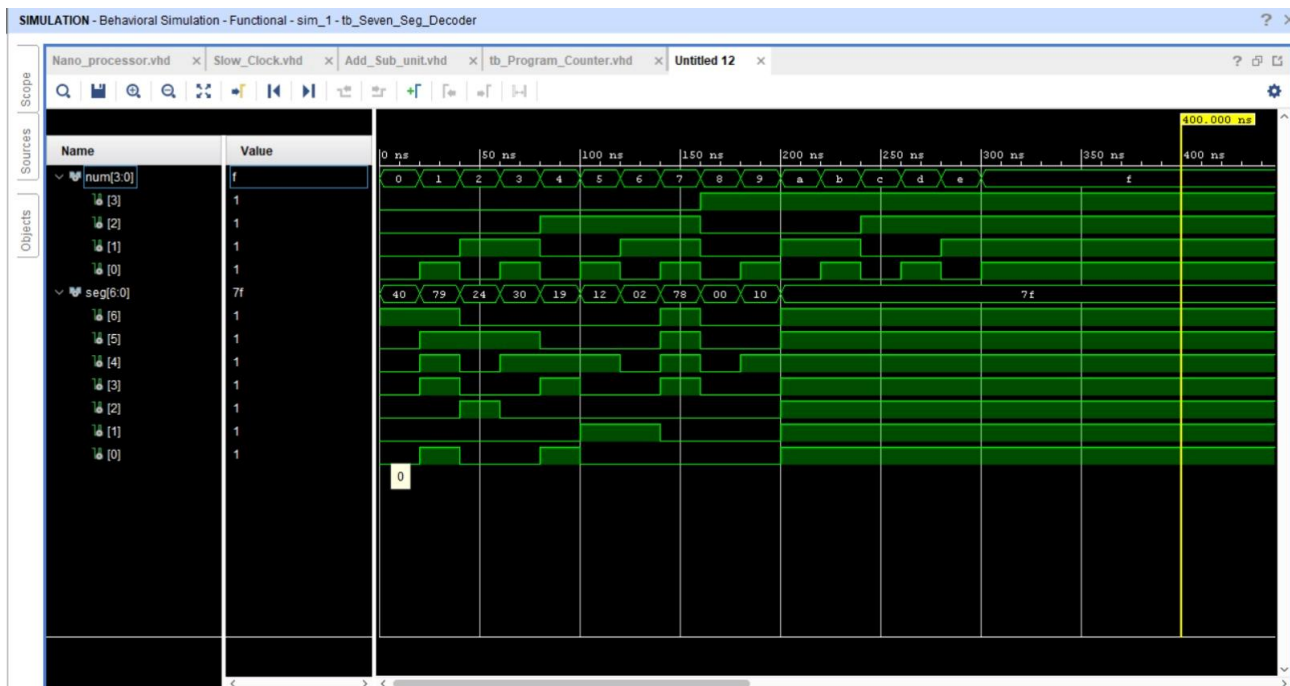
num <= "1111"; wait for 20 ns;
assert seg = "1111111" report "FAIL: digit F (blank)" severity error;

report "tb_Seven_Seg_Decoder: ALL TESTS DONE" severity note;
wait;
end process;

end Behavioral;

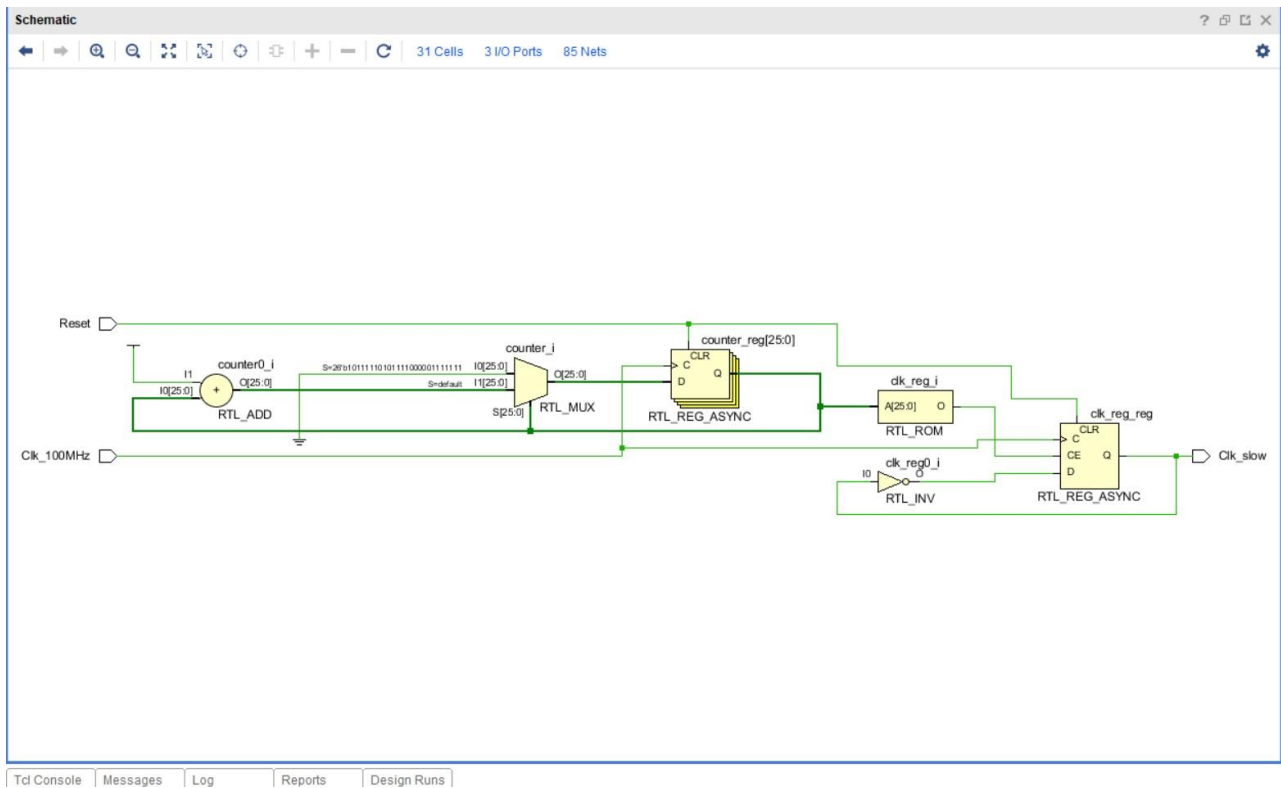
```

Timing Diagram



Slow_Clock.vhd

Schematic Diagram



Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Slow_Clock is
    Port ( Clk_100MHz : in  STD_LOGIC;
          Reset      : in  STD_LOGIC;
          Clk_slow   : out STD_LOGIC);
end Slow_Clock;

architecture Behavioral of Slow_Clock is
    -- 50_000_000 counts at 100MHz = 0.5s half-period = 1 Hz clock
    -- Change to 25_000_000 for 2 Hz (faster, easier to debug)
    constant MAX_COUNT : integer := 50000000;
    signal  counter      : integer range 0 to 50000000 := 0;
    signal  clk_reg      : STD_LOGIC := '0';
begin
    process(Clk_100MHz, Reset)
    begin
        if Reset = '1' then
            counter <= 0;
            clk_reg <= '0';
        elsif rising_edge(Clk_100MHz) then
            if counter = MAX_COUNT - 1 then
                counter <= 0;
                clk_reg <= not clk_reg;
            end if;
        end if;
    end process;
end Behavioral;
```

```

        else
            counter <= counter + 1;
        end if;
    end if;
end process;
Clk_slow <= clk_reg;
end Behavioral;

```

Simulation Source Code

```

-----
--
-- Testbench : tb_Slow_Clock
-- DUT       : Slow_Clock
--
-- IMPORTANT: Before simulating, temporarily change MAX_COUNT in Slow_Clock
-- from 50_000_000 to a small value, e.g. 4, so the simulation finishes fast:
--
--     constant MAX_COUNT : integer := 4;    -- for simulation only
--
-- This testbench verifies:
-- 1. Reset holds Clk_slow = '0'
-- 2. Clk_slow stays '0' for MAX_COUNT cycles then toggles
-- 3. Clk_slow toggles again after another MAX_COUNT cycles
-- 4. Re-asserting Reset immediately clears Clk_slow
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Slow_Clock is
end tb_Slow_Clock;

architecture Behavioral of tb_Slow_Clock is

    component Slow_Clock
        Port ( Clk_100MHz : in  STD_LOGIC;
              Reset      : in  STD_LOGIC;
              Clk_slow   : out STD_LOGIC);
    end component;

    signal Clk_100MHz : STD_LOGIC := '0';
    signal Reset      : STD_LOGIC := '0';
    signal Clk_slow   : STD_LOGIC;

    -- Match whatever MAX_COUNT you set in the DUT for simulation
    -- If DUT has MAX_COUNT=4, one half-period = 4 * 10ns = 40ns
    constant CLK_PERIOD      : time := 10 ns;    -- 100 MHz
    constant MAX_CNT         : integer := 4;     -- must match DUT's MAX_COUNT
    constant HALF_SLOW       : time := MAX_CNT * CLK_PERIOD;

begin

    DUT: Slow_Clock port map(
        Clk_100MHz => Clk_100MHz,
        Reset      => Reset,
        Clk_slow   => Clk_slow);

    -- 100 MHz clock
    Clk_100MHz <= not Clk_100MHz after CLK_PERIOD / 2;

```

```

process
begin
    -- Test 1: Reset holds Clk_slow = 0
    Reset <= '1'; wait for 5 * CLK_PERIOD;
    assert Clk_slow = '0' report "FAIL: Reset should hold Clk_slow=0"
severity error;
    Reset <= '0';

    -- Test 2: Clk_slow starts low; after MAX_COUNT rising edges it goes
high
    assert Clk_slow = '0' report "FAIL: Should start at 0 after reset"
severity error;
    wait for HALF_SLOW + CLK_PERIOD;    -- one half-period + margin
    assert Clk_slow = '1' report "FAIL: Clk_slow should toggle to 1"
severity error;

    -- Test 3: Another MAX_COUNT cycles â†' toggles back to 0
    wait for HALF_SLOW;
    assert Clk_slow = '0' report "FAIL: Clk_slow should toggle back to 0"
severity error;

    -- Test 4: One more toggle
    wait for HALF_SLOW;
    assert Clk_slow = '1' report "FAIL: Clk_slow should toggle to 1 again"
severity error;

    -- Test 5: Reset mid-operation clears immediately
    Reset <= '1'; wait for CLK_PERIOD;
    assert Clk_slow = '0' report "FAIL: Mid-run reset" severity error;
    Reset <= '0';

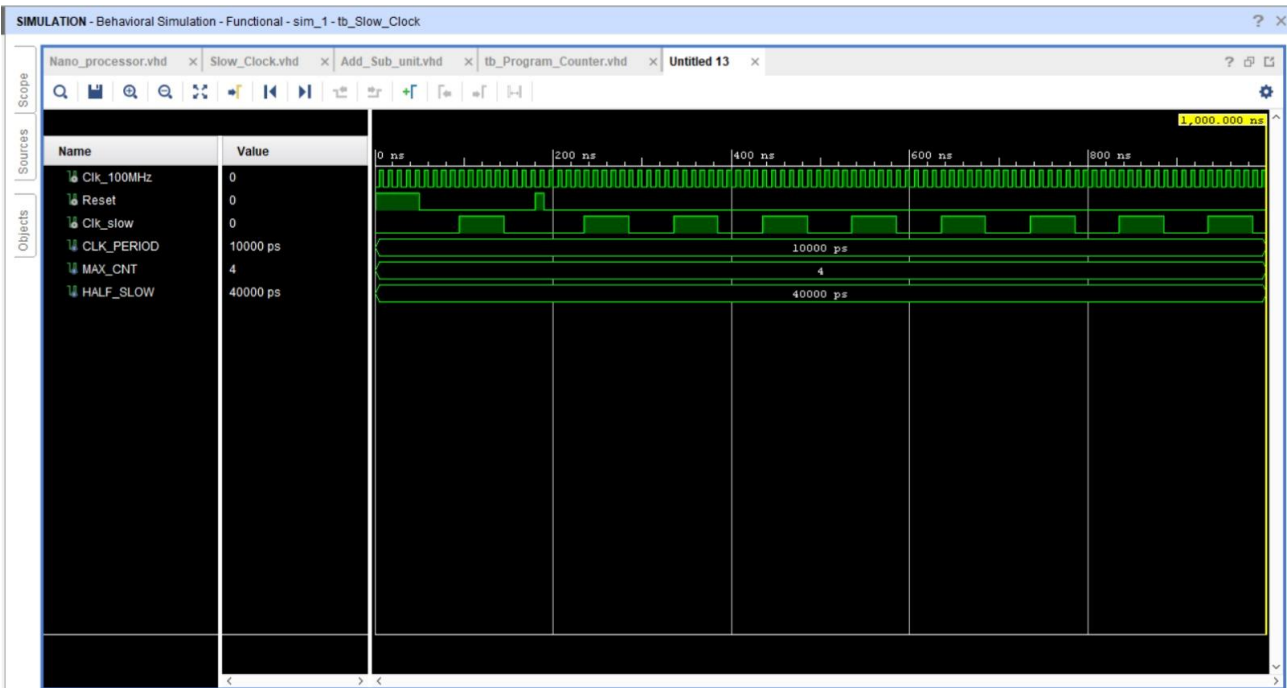
    -- Test 6: Resumes toggling from 0 after reset
    wait for HALF_SLOW + CLK_PERIOD;
    assert Clk_slow = '1' report "FAIL: Clk_slow should resume toggling
after reset" severity error;

    report "tb_Slow_Clock: ALL TESTS DONE" severity note;
    wait;
end process;

end Behavioral;

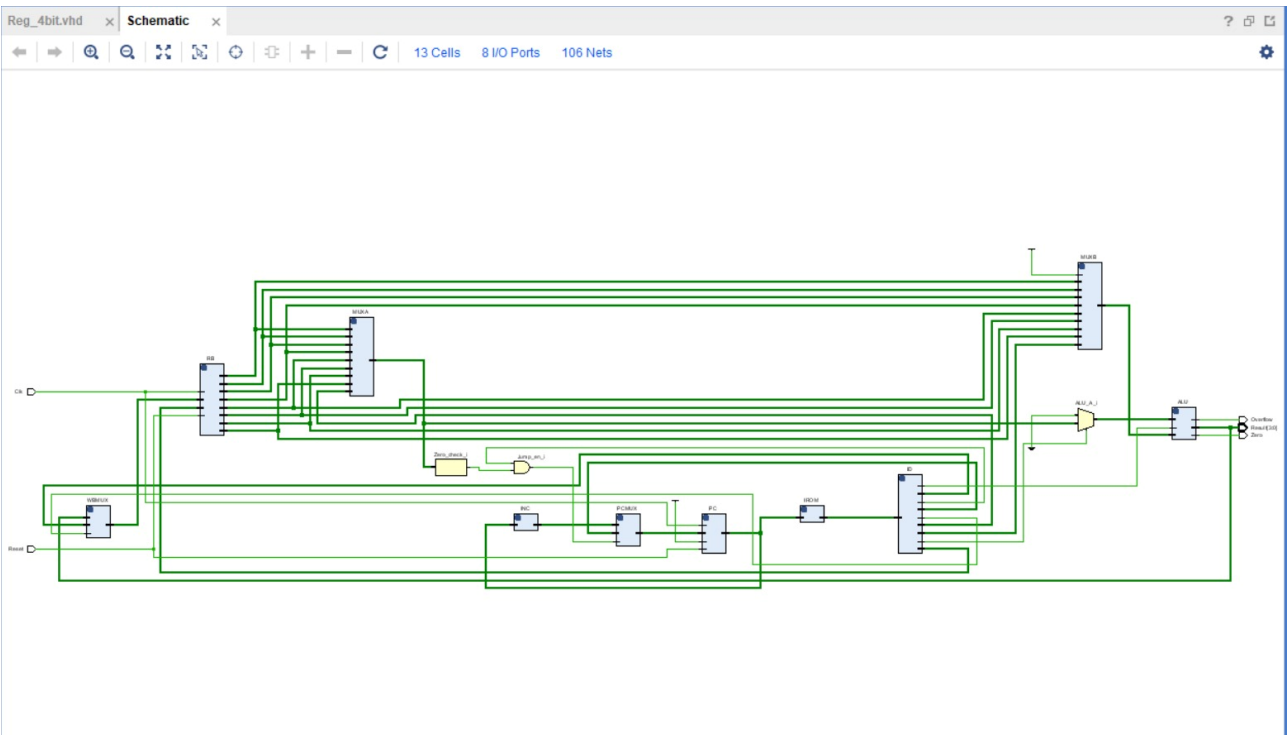
```

Timing Diagram



Nano_processor.vhd — top-level

Schematic Diagram



Design Source Code

```
-- Company:
-- Engineer:
--
-- Create Date: 04/29/2026 02:44:04 PM
-- Design Name:
-- Module Name: Nano_processor - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;
```

```
-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity Nano_processor is
    Port ( Clk      : in  STD_LOGIC;
          Reset    : in  STD_LOGIC;
          Result   : out STD_LOGIC_VECTOR(3 downto 0);
          Overflow : out STD_LOGIC;
          Zero     : out STD_LOGIC);
end Nano_processor;
```

```
architecture Structural of Nano_processor is
```

```
    -- Component declarations
    component Program_Counter
        port(D:in STD_LOGIC_VECTOR(2 downto 0); Load,Reset,Clk:in STD_LOGIC;
             Q:out STD_LOGIC_VECTOR(2 downto 0));
    end component;
    component pc_incrementer
        port(A:in STD_LOGIC_VECTOR(2 downto 0);
             Sum:out STD_LOGIC_VECTOR(2 downto 0); Cout:out STD_LOGIC);
    end component;
    component Mux_2_3bit
        port(I0,I1:in STD_LOGIC_VECTOR(2 downto 0); Sel:in STD_LOGIC;
             Y:out STD_LOGIC_VECTOR(2 downto 0));
    end component;
    component ROM
        port(address:in STD_LOGIC_VECTOR(2 downto 0);
```

```

        data:out STD_LOGIC_VECTOR(11 downto 0));
end component;
component Instruction_decoder
    port(Instruction:in STD_LOGIC_VECTOR(11 downto 0);
        Reg_En:out STD_LOGIC_VECTOR(7 downto 0);
        Mux_Sel_A,Mux_Sel_B,Jump_Addr:out STD_LOGIC_VECTOR(2 downto 0);
        Imm_Val:out STD_LOGIC_VECTOR(3 downto 0);
        Load_Imm,Add_Sub,Neg_En,Jump:out STD_LOGIC);
end component;
component Registers_Bank
    port(D:in STD_LOGIC_VECTOR(3 downto 0);
        Reg_En:in STD_LOGIC_VECTOR(7 downto 0);
        Clk,Reset:in STD_LOGIC;
        Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7:out STD_LOGIC_VECTOR(3 downto 0));
end component;
component Mux_8_4bit
    port(I0,I1,I2,I3,I4,I5,I6,I7:in STD_LOGIC_VECTOR(3 downto 0);
        Sel:in STD_LOGIC_VECTOR(2 downto 0);
        Y:out STD_LOGIC_VECTOR(3 downto 0));
end component;
component Mux_8_4bit_new
    port(I0,I1,I2,I3,I4,I5,I6,I7:in STD_LOGIC_VECTOR(3 downto 0);
        Sel:in STD_LOGIC_VECTOR(2 downto 0);
        Enable:in STD_LOGIC;
        Y:out STD_LOGIC_VECTOR(3 downto 0));
end component;
component Add_Sub_unit
    port(A,B:in STD_LOGIC_VECTOR(3 downto 0);
        Add_Sub:in STD_LOGIC;
        Result:out STD_LOGIC_VECTOR;
        Overflow:out STD_LOGIC;
        Zero:out STD_LOGIC);
end component;
component Mux_2_4bit
    port(I0,I1:in STD_LOGIC_VECTOR(3 downto 0); Sel:in STD_LOGIC;
        Y:out STD_LOGIC_VECTOR(3 downto 0));
end component;

```

```

-- Internal signals
signal PC_out, PC_inc, PC_in, Jump_Addr : STD_LOGIC_VECTOR(2 downto 0);
signal instruction                       : STD_LOGIC_VECTOR(11 downto 0);
signal Reg_En                           : STD_LOGIC_VECTOR(7 downto 0);
signal Q0,Q1,Q2,Q3,Q4,Q5,Q6,Q7         : STD_LOGIC_VECTOR(3 downto 0);
signal Mux_Sel_A, Mux_Sel_B             : STD_LOGIC_VECTOR(2 downto 0);
signal ALU_A_raw, ALU_A, ALU_B          : STD_LOGIC_VECTOR(3 downto 0);
signal ALU_Result, Imm_Val, Reg_In      : STD_LOGIC_VECTOR(3 downto 0);
signal Overflow_sig                     : STD_LOGIC;
signal Load_Imm, Add_Sub, Neg_En        : STD_LOGIC;
signal Jump, Zero_sig, Cout_PC          : STD_LOGIC;
signal Zero_check, Jump_en              : STD_LOGIC;

```

begin

```

-- 1. Program Counter
PC: Program_Counter port map(
    D=>PC_in, Load=>'1', Reset=>Reset, Clk=>Clk, Q=>PC_out);

-- 2. PC Incrementer (PC + 1)
INC: pc_incrementer port map(A=>PC_out, Sum=>PC_inc, Cout=>Cout_PC);

-- 3. Next PC MUX: PC+1 or jump address
Jump_en <= Jump and Zero_check;
PCMUX: Mux_2_3bit port map(
    I0=>PC_inc, I1=>Jump_Addr, Sel=>Jump_en, Y=>PC_in);

```

```

-- 4. ROM
IROM: ROM port map(address=>PC_out, data=>instruction);

-- 5. Instruction Decoder
ID: Instruction_decoder port map(
    Instruction=>instruction, Reg_En=>Reg_En,
    Mux_Sel_A=>Mux_Sel_A, Mux_Sel_B=>Mux_Sel_B,
    Imm_Val=>Imm_Val, Load_Imm=>Load_Imm,
    Add_Sub=>Add_Sub, Neg_En=>Neg_En,
    Jump=>Jump, Jump_Addr=>Jump_Addr);

-- 6. Register Bank
RB: Registers_Bank port map(
    D=>Reg_In, Reg_En=>Reg_En, Clk=>Clk, Reset=>Reset,
    Q0=>Q0, Q1=>Q1, Q2=>Q2, Q3=>Q3,
    Q4=>Q4, Q5=>Q5, Q6=>Q6, Q7=>Q7);

-- 7. MUX for ALU input A (Rs1)
MUXA: Mux_8_4bit port map(
    I0=>Q0, I1=>Q1, I2=>Q2, I3=>Q3,
    I4=>Q4, I5=>Q5, I6=>Q6, I7=>Q7,
    Sel=>Mux_Sel_A, Y=>ALU_A_raw);

-- 8. Force ALU_A = 0 for NEG operation
ALU_A <= "0000" when Neg_En = '1' else ALU_A_raw;

-- 9. MUX for ALU input B (Rs2, disabled=0000 for NEG)
MUXB: Mux_8_4bit_new port map(
    I0=>Q0, I1=>Q1, I2=>Q2, I3=>Q3,
    I4=>Q4, I5=>Q5, I6=>Q6, I7=>Q7,
    Sel=>Mux_Sel_B, Enable=>'1', Y=>ALU_B);

-- 10. ALU
ALU: Add_Sub_unit port map(
    A=>ALU_A, B=>ALU_B, Add_Sub=>Add_Sub,
    Result=>ALU_Result, Overflow=>Overflow_sig,
    Zero=>Zero_sig);

-- 11. Write-back MUX: ALU result or immediate
WBMUX: Mux_2_4bit port map(
    I0=>ALU_Result, I1=>Imm_Val,
    Sel=>Load_Imm, Y=>Reg_In);

-- 12. Zero check for JZR (checks Rs value on ALU_A_raw)
Zero_check <= '1' when ALU_A_raw = "0000" else '0';

-- Outputs
Result    <= ALU_Result;
Overflow   <= Overflow_sig;
Zero       <= Zero_sig;
end Structural;

```

Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- =====
-- Nano_processor_tb

```



```

-- Testbench for the corrected lab-spec Nano_processor
-- Program in ROM computes 1+2+3 = 6, stores result in R7
--
-- Instruction encoding (2-bit opcode):
--   MOVI R,d   : 10 RRR 000 dddd
--   ADD Ra,Rb  : 00 RaRaRa RbRbRb 0000
--   NEG R      : 01 RRR 0000000000
--   JZR R,d    : 11 RRR 0000 ddd
--
-- ROM program:
--   addr 0: MOVI R1, #1      R1 = 1
--   addr 1: MOVI R2, #2      R2 = 2
--   addr 2: MOVI R3, #3      R3 = 3
--   addr 3: ADD  R7, R1      R7 = 0 + 1 = 1
--   addr 4: ADD  R7, R2      R7 = 1 + 2 = 3
--   addr 5: ADD  R7, R3      R7 = 3 + 3 = 6   final answer
--   addr 6: NOP
--   addr 7: NOP
-- =====

```

```

entity Nano_processor_tb is
end Nano_processor_tb;

```

```

architecture Behavioral of Nano_processor_tb is

```

```

-- Component under test
component Nano_processor
    Port ( Clk      : in  STD_LOGIC;
          Reset     : in  STD_LOGIC;
          R7_out    : out STD_LOGIC_VECTOR(3 downto 0);
          Overflow   : out STD_LOGIC;
          Zero       : out STD_LOGIC;
          seg        : out STD_LOGIC_VECTOR(6 downto 0);
          an         : out STD_LOGIC_VECTOR(3 downto 0));
end component;

```

```

-- Testbench signals
signal Clk      : STD_LOGIC := '0';
signal Reset    : STD_LOGIC := '1';
signal R7_out   : STD_LOGIC_VECTOR(3 downto 0);
signal Overflow : STD_LOGIC;
signal Zero     : STD_LOGIC;
signal seg      : STD_LOGIC_VECTOR(6 downto 0);
signal an       : STD_LOGIC_VECTOR(3 downto 0);

```

```

-- Clock period
-- We use a FAST clock in simulation so we don't have to wait
-- 50 million cycles for the slow clock divider.
-- The Slow_Clock inside Nano_processor uses MAX_COUNT=50_000_000
-- at 100MHz. In simulation we set CLK_PERIOD = 10ns (100MHz) and
-- the slow clock fires every 500ms of sim time that is fine for
-- behavioural sim. To speed things up, temporarily change
-- MAX_COUNT in Slow_Clock.vhd to 5 before simulating (then restore).
--
-- SIMULATION TIP: Change Slow_Clock MAX_COUNT to 5 for fast sim:
--   constant MAX_COUNT : integer := 5;
-- This makes the slow clock toggle every 5 fast cycles.
-- Restore to 50_000_000 before generating bitstream.
--

```

```

=====
=====

```

```

    constant CLK_PERIOD : time := 10 ns; -- 100 MHz fast clock

```

```

-- Helper: convert SLV to integer string

```

```

function to_str(s : STD_LOGIC_VECTOR) return string is
begin
    return integer'image(to_integer(unsigned(s)));
end function;

-- Helper: 4-bit SLV to binary string
function to_bin(s : STD_LOGIC_VECTOR(3 downto 0)) return string is
    variable result : string(1 to 4);
begin
    for i in 3 downto 0 loop
        if s(i) = '1' then
            result(4-i) := '1';
        else
            result(4-i) := '0';
        end if;
    end loop;
    return result;
end function;

begin

--Instantiate UUT
UUT: Nano_processor
    port map (
        Clk      => Clk,
        Reset    => Reset,
        R7_out   => R7_out,
        Overflow  => Overflow,
        Zero     => Zero,
        seg      => seg,
        an       => an);

-- Clock generator (100 MHz)
clk_proc: process
begin
    Clk <= '0'; wait for CLK_PERIOD / 2;
    Clk <= '1'; wait for CLK_PERIOD / 2;
end process;

=====
-- STIMULUS PROCESS
-- NOTE: With Slow_Clock MAX_COUNT=5 (simulation mode),
--       each slow clock cycle = 10 fast cycles = 100 ns.
--       With MAX_COUNT=50_000_000 (board mode), each
--       slow cycle = 1 second of simulation time.
-- =====
stim_proc: process

    -- Wait for exactly N slow clock cycles
    -- Each slow clock half-period = MAX_COUNT * CLK_PERIOD
    -- For MAX_COUNT=5: slow half = 50ns, full = 100ns
    -- Adjust SLOW_CLK_PERIOD to match your MAX_COUNT setting
    constant SLOW_CLK_PERIOD : time := 100 ns; -- for MAX_COUNT=5
    constant SLOW_CLK_PERIOD : time := 1000 ms; -- for
MAX_COUNT=50_000_000

begin

    -- RESET
=====
Reset <= '1';
report "===== " severity note;
report " Nano_processor Testbench START" severity note;

```

```

report " Program: compute 1+2+3 = 6 in R7" severity note;
report "===== " severity note;
wait for CLK_PERIOD * 4;
Reset <= '0';
wait for CLK_PERIOD * 2;
report "[RESET released] Processor starting..." severity note;

-- CYCLE 1: MOVI R1, #1
-- Instruction: 10 001 000 0001 = "100010000001"
-- R1 <= 1
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 1] MOVI R1, #1" severity note;
report " R7_out = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
report " Overflow = " & STD_LOGIC'image(Overflow) severity note;
report " Zero = " & STD_LOGIC'image(Zero) severity note;
-- R7 still 0 at this point (nothing written to R7 yet)
assert R7_out = "0000"
    report "[FAIL] Cycle 1: R7 should still be 0, got " & to_str(R7_out)
    severity error;

-- CYCLE 2: MOVI R2, #2
-- Instruction: 10 010 000 0010 = "100100000010"
-- R2 <= 2
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 2] MOVI R2, #2" severity note;
report " R7_out = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
assert R7_out = "0000"
    report "[FAIL] Cycle 2: R7 should still be 0, got " & to_str(R7_out)
    severity error;

-- CYCLE 3: MOVI R3, #3
-- Instruction: 10 011 000 0011 = "100110000011"
-- R3 <= 3
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 3] MOVI R3, #3" severity note;
report " R7_out = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
assert R7_out = "0000"
    report "[FAIL] Cycle 3: R7 should still be 0, got " & to_str(R7_out)
    severity error;

-- CYCLE 4: ADD R7, R1
-- Instruction: 00 111 001 0000 = "001110010000"
-- R7 = R7 + R1 = 0 + 1 = 1
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 4] ADD R7, R1 R7 = 0 + 1 = 1" severity note;
report " R7_out = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
report " Overflow = " & STD_LOGIC'image(Overflow) severity note;
report " Zero = " & STD_LOGIC'image(Zero) severity note;
assert R7_out = "0001"
    report "[FAIL] Cycle 4: ADD R7,R1 expected 1 (0001), got " &
to_str(R7_out)
    severity error;
assert Overflow = '0'
    report "[FAIL] Cycle 4: Overflow should be 0" severity error;
assert Zero = '0'
    report "[FAIL] Cycle 4: Zero should be 0 (result=1)" severity error;

```

```

-- CYCLE 5: ADD R7, R2
-- Instruction: 00 111 010 0000 = "001110100000"
-- R7 = R7 + R2 = 1 + 2 = 3
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 5] ADD R7, R2  R7 = 1 + 2 = 3" severity note;
report "  R7_out   = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
report "  Overflow = " & STD_LOGIC'image(Overflow) severity note;
report "  Zero      = " & STD_LOGIC'image(Zero) severity note;
assert R7_out = "0011"
  report "[FAIL] Cycle 5: ADD R7,R2 expected 3 (0011), got " &
to_str(R7_out)
  severity error;
assert Overflow = '0'
  report "[FAIL] Cycle 5: Overflow should be 0" severity error;

-- CYCLE 6: ADD R7, R3
-- Instruction: 00 111 011 0000 = "001110110000"
-- R7 = R7 + R3 = 3 + 3 = 6 = "0110"  â†? FINAL ANSWER
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 6] ADD R7, R3  R7 = 3 + 3 = 6  *** FINAL RESULT ***"
severity note;
report "  R7_out   = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
report "  Overflow = " & STD_LOGIC'image(Overflow) severity note;
report "  Zero      = " & STD_LOGIC'image(Zero) severity note;
assert R7_out = "0110"
  report "[FAIL] Cycle 6: ADD R7,R3 expected 6 (0110), got " &
to_str(R7_out)
  severity error;
assert Overflow = '0'
  report "[FAIL] Cycle 6: Overflow should be 0 (6 fits in 4 bits)"
severity error;
assert Zero = '0'
  report "[FAIL] Cycle 6: Zero should be 0 (result=6)" severity error;
report "  7-seg = " & STD_LOGIC'image(seg(6)) &
  STD_LOGIC'image(seg(5)) &
  STD_LOGIC'image(seg(4)) &
  STD_LOGIC'image(seg(3)) &
  STD_LOGIC'image(seg(2)) &
  STD_LOGIC'image(seg(1)) &
  STD_LOGIC'image(seg(0)) &
  " (expect 0000010 = digit 6)" severity note;
assert seg = "0000010"
  report "[FAIL] Cycle 6: 7-seg expected '6' (0000010), got different"
  severity error;

-- CYCLE 7: NOP
wait for SLOW_CLK_PERIOD;
report "-----" severity note;
report "[Cycle 7] NOP  R7 unchanged = 6" severity note;
report "  R7_out   = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
assert R7_out = "0110"
  report "[FAIL] Cycle 7: NOP should leave R7=6 (0110), got " &
to_str(R7_out)
  severity error;

-- CYCLE 8: NOP
wait for SLOW_CLK_PERIOD;
report "[Cycle 8] NOP  R7 unchanged = 6" severity note;

```

```

    assert R7_out = "0110"
        report "[FAIL] Cycle 8: NOP should leave R7=6 (0110), got " &
to_str(R7_out)
        severity error;

-- TEST RESET
report "-----" severity note;
report "[TEST] Applying Reset mid-execution..." severity note;
Reset <= '1';
wait for SLOW_CLK_PERIOD;
report "  R7_out after Reset = " & to_bin(R7_out) & " (" &
to_str(R7_out) & ")" severity note;
assert R7_out = "0000"
    report "[FAIL] Reset: R7 should be 0000 after reset, got " &
to_str(R7_out)
    severity error;

-- RE-RUN after Reset
report "-----" severity note;
report "[TEST] Releasing Reset  processor re-executes..." severity note;
Reset <= '0';
wait for SLOW_CLK_PERIOD * 7; -- wait through all 6 active instructions
+ 1

report "-----" severity note;
report "[TEST] After full re-run: checking R7 = 6" severity note;
report "  R7_out    = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
assert R7_out = "0110"
    report "[FAIL] Re-run: R7 expected 6 (0110) after second run, got "
& to_str(R7_out)
    severity error;

-- SUMMARY
report "===== " severity note;
report " Nano_processor Testbench COMPLETE" severity note;
report " Expected: R7 = 6 (0110) = 1+2+3" severity note;
report " Got:      R7 = " & to_bin(R7_out) & " (" & to_str(R7_out) & ")"
severity note;
report "===== " severity note;

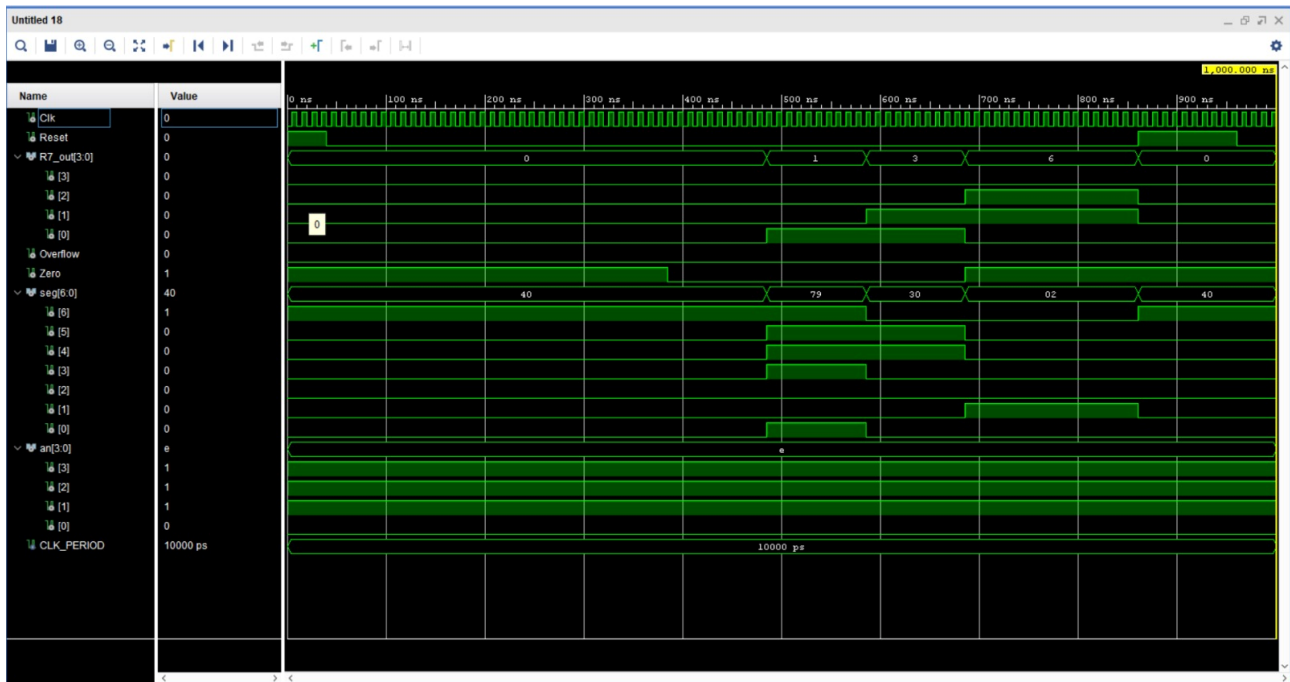
wait; -- stop simulation
end process;

--
-- MONITOR PROCESS
-- Prints R7, Overflow, Zero on every slow clock rising edge
-- (monitors the slow clock signal indirectly via periodic check)
--
monitor_proc: process(Clk)
    variable cycle_cnt : integer := 0;
begin
    if rising_edge(Clk) and Reset = '0' then
        cycle_cnt := cycle_cnt + 1;
        -- Print every 10 fast clock cycles (adjust for your MAX_COUNT)
        if cycle_cnt mod 10 = 0 then
            report "FAST_CLK#" & integer'image(cycle_cnt) &
                " | R7=" & to_bin(R7_out) &
                " OVF=" & STD_LOGIC'image(Overflow) &
                " Z=" & STD_LOGIC'image(Zero)
                severity note;
            end if;
        end if;
    end process;
end process;

```

```
end Behavioral;
```

Timing Diagram



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Comparator is
    Port ( A      : in  STD_LOGIC_VECTOR(3 downto 0);
          B      : in  STD_LOGIC_VECTOR(3 downto 0);
          GT      : out STD_LOGIC;      -- A > B
          LT      : out STD_LOGIC;      -- A < B
          EQ      : out STD_LOGIC);     -- A = B
end Comparator;
```

```

architecture Behavioral of Comparator is
begin
    process(A, B)
    begin
        if unsigned(A) > unsigned(B) then
            GT <= '1'; LT <= '0'; EQ <= '0';
        elsif unsigned(A) < unsigned(B) then
            GT <= '0'; LT <= '1'; EQ <= '0';
        else
            GT <= '0'; LT <= '0'; EQ <= '1';
        end if;
    end process;
end Behavioral;

```

Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Comparator is
end TB_Comparator;

architecture Behavioral of TB_Comparator is

    component Comparator
        Port (
            A      : in  STD_LOGIC_VECTOR(3 downto 0);
            B      : in  STD_LOGIC_VECTOR(3 downto 0);
            EQ      : out STD_LOGIC;
            GT      : out STD_LOGIC;
            LT      : out STD_LOGIC
        );
    end component;

    signal A, B : STD_LOGIC_VECTOR(3 downto 0);
    signal EQ, GT, LT : STD_LOGIC;

begin

    UUT: Comparator
        port map (A => A, B => B, EQ => EQ, GT => GT, LT => LT);

    process
    begin
        -- A = B
        A <= "0101"; B <= "0101"; wait for 20 ns;
        assert EQ = '1' and GT = '0' and LT = '0'
            report "FAIL: A=B=5" severity error;

        -- A > B
        A <= "1001"; B <= "0011"; wait for 20 ns;
        assert EQ = '0' and GT = '1' and LT = '0'
            report "FAIL: A>B (9>3)" severity error;

        -- A < B
        A <= "0010"; B <= "1110"; wait for 20 ns;
        assert EQ = '0' and GT = '0' and LT = '1'
            report "FAIL: A<B (2<14)" severity error;

        -- A = 0, B = 0
        A <= "0000"; B <= "0000"; wait for 20 ns;
    end process;
end Behavioral;

```



```

assert EQ = '1' and GT = '0' and LT = '0'
    report "FAIL: A=B=0" severity error;

-- A = 15, B = 0
A <= "1111"; B <= "0000"; wait for 20 ns;
assert GT = '1' report "FAIL: A=15 > B=0" severity error;

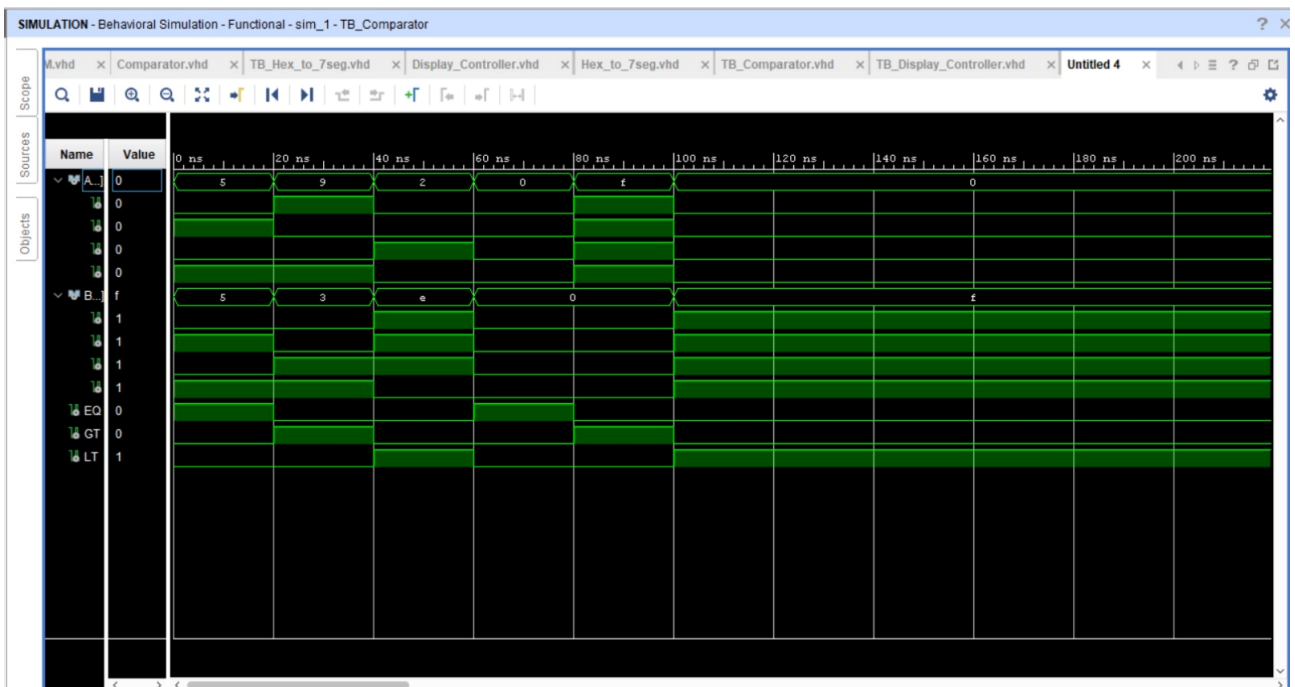
-- A = 0, B = 15
A <= "0000"; B <= "1111"; wait for 20 ns;
assert LT = '1' report "FAIL: A=0 < B=15" severity error;

report "TB_Comparator: All tests passed." severity note;
wait;
end process;

end Behavioral;

```

Timing Diagram



```
-- Module Name: Display_Controller - Behavioral
-- Description: 4-digit time-multiplexed 7-segment display controller.
--               Refreshes all 4 digits at ~1 kHz each (4 kHz total scan).
--
-- Digit layout on Basys3 (left to right):
--
-- â"â"â"â"â"â"â"â"â"â"â"â"â"â"â"â"â" â"¡â"¢â"£â"¤â"¥â"¦â"§â"¨â"©â"ªâ"«â"¬â"­â"®â"¯â"°â"±â"²â"³â"´â"µâ"¶â"·â"¸â"¹â"ºâ"»â"¼â"½â"¾â"¿
--   â", Digit 3 â", Digit 2 â", Digit 1 â", Digit 0 â",
--   â", PC[2:0]â", GT/LT  â",OVF/ZERO â", Result  â",
--
-- â"â"â"â"â"â"â"â"â"â"â"â"â"â"â"â"â" â"¡â"¢â"£â"¤â"¥â"¦â"§â"¨â"©â"ªâ"«â"¬â"­â"®â"¯â"°â"±â"²â"³â"´â"µâ"¶â"·â"¸â"¹â"ºâ"»â"¼â"½â"¾â"¿
--   an[3]      an[2]      an[1]      an[0]
--
-- Digit 0 (an[0]) : Result[3:0]           hex digit e.g. "E" for 14
-- Digit 1 (an[1]) : flags OVF/Zero       shows "0F" style
--                   upper nibble: {0,0,0,Overflow}
--                   lower nibble: {0,0,0,Zero}         -- but we encode as hex
--                   Actually encoded as: {2'b0, Overflow, Zero} -> 0..3
-- Digit 2 (an[2]) : comparator flags     {2'b0, GT, LT} -> 0..3
-- Digit 3 (an[3]) : PC address[2:0]      0..7
--
-- Refresh: with 100 MHz clock, divide by 25000 -> 4 kHz scan rate
--          -> each digit gets ~1 ms on time (flicker-free)
```

```

-----
--

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display_Controller is
    Port ( Clk      : in  STD_LOGIC;
           Reset    : in  STD_LOGIC;
           -- Data inputs
           Result    : in  STD_LOGIC_VECTOR(3 downto 0);    -- digit 0
           Overflow  : in  STD_LOGIC;                        -- digit 1 bit1
           Zero      : in  STD_LOGIC;                        -- digit 1 bit0
           GT        : in  STD_LOGIC;                        -- digit 2 bit1
           LT        : in  STD_LOGIC;                        -- digit 2 bit0
           PC        : in  STD_LOGIC_VECTOR(2 downto 0);    -- digit 3
           -- 7-segment outputs (active low)
           seg       : out STD_LOGIC_VECTOR(6 downto 0);
           an        : out STD_LOGIC_VECTOR(3 downto 0));
end Display_Controller;

architecture Behavioral of Display_Controller is

    -- Refresh counter: 100 MHz / 25000 = 4000 Hz scan rate
    constant REFRESH_MAX : integer := 24999;
    signal refresh_cnt    : integer range 0 to REFRESH_MAX := 0;
    signal digit_sel      : STD_LOGIC_VECTOR(1 downto 0) := "00";

    -- Active digit data
    signal active_hex     : STD_LOGIC_VECTOR(3 downto 0);

    -- Decoder
    component Hex_to_7seg
        Port ( hex_in : in  STD_LOGIC_VECTOR(3 downto 0);
              seg     : out STD_LOGIC_VECTOR(6 downto 0));
    end component;

begin

    -- -----
    -- Refresh counter & digit selector
    -- -----
    process(Clk, Reset)
    begin
        if Reset = '1' then
            refresh_cnt <= 0;
            digit_sel   <= "00";
        elsif rising_edge(Clk) then
            if refresh_cnt = REFRESH_MAX then
                refresh_cnt <= 0;
                digit_sel   <= std_logic_vector(unsigned(digit_sel) + 1);
            else
                refresh_cnt <= refresh_cnt + 1;
            end if;
        end if;
    end process;

    -- -----
    -- Select which digit's data to display & drive anode
    -- -----
    process(digit_sel, Result, Overflow, Zero, GT, LT, PC)
    begin
        case digit_sel is

```

```

        when "00" => -- Digit 0: Result (hex)
            active_hex <= Result;
            an          <= "1110";    -- an[0] active (low)

        when "01" => -- Digit 1: OVF & Zero flags
            -- Encode as: bit1=Overflow, bit0=Zero -> shows 0,1,2,3
            active_hex <= "00" & Overflow & Zero;
            an          <= "1101";    -- an[1] active

        when "10" => -- Digit 2: GT & LT flags
            -- Encode as: bit1=GT, bit0=LT -> shows 0,1,2,3
            active_hex <= "00" & GT & LT;
            an          <= "1011";    -- an[2] active

        when others => -- Digit 3: PC address (0-7)
            active_hex <= '0' & PC;
            an          <= "0111";    -- an[3] active
    end case;
end process;

-- -----
-- Hex decoder
-- -----

DECODER: Hex_to_7seg
    port map (hex_in => active_hex, seg => seg);

end Behavioral;

```

Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Display_Controller is
end TB_Display_Controller;

architecture Behavioral of TB_Display_Controller is

    component Display_Controller
        Port (
            Clk          : in  STD_LOGIC;
            Reset         : in  STD_LOGIC;
            Data          : in  STD_LOGIC_VECTOR(7 downto 0);
            Anode         : out STD_LOGIC_VECTOR(3 downto 0);
            Cathode       : out STD_LOGIC_VECTOR(6 downto 0)
        );
    end component;

    signal Clk          : STD_LOGIC := '0';
    signal Reset        : STD_LOGIC := '0';
    signal Data         : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal Anode        : STD_LOGIC_VECTOR(3 downto 0);
    signal Cathode      : STD_LOGIC_VECTOR(6 downto 0);

    constant CLK_PERIOD : time := 10 ns;

begin

    Clk <= not Clk after CLK_PERIOD / 2;

```

```

UUT: Display_Controller
  port map (Clk => Clk, Reset => Reset, Data => Data,
           Anode => Anode, Cathode => Cathode);

process
begin
  Reset <= '1'; wait for 5 * CLK_PERIOD;
  Reset <= '0';

  -- Display 0xAB = 171
  Data <= "10101011"; wait for 500 * CLK_PERIOD;

  -- Display 0x00
  Data <= "00000000"; wait for 500 * CLK_PERIOD;

  -- Display 0xFF
  Data <= "11111111"; wait for 500 * CLK_PERIOD;

  -- Display 0x42
  Data <= "01000010"; wait for 500 * CLK_PERIOD;

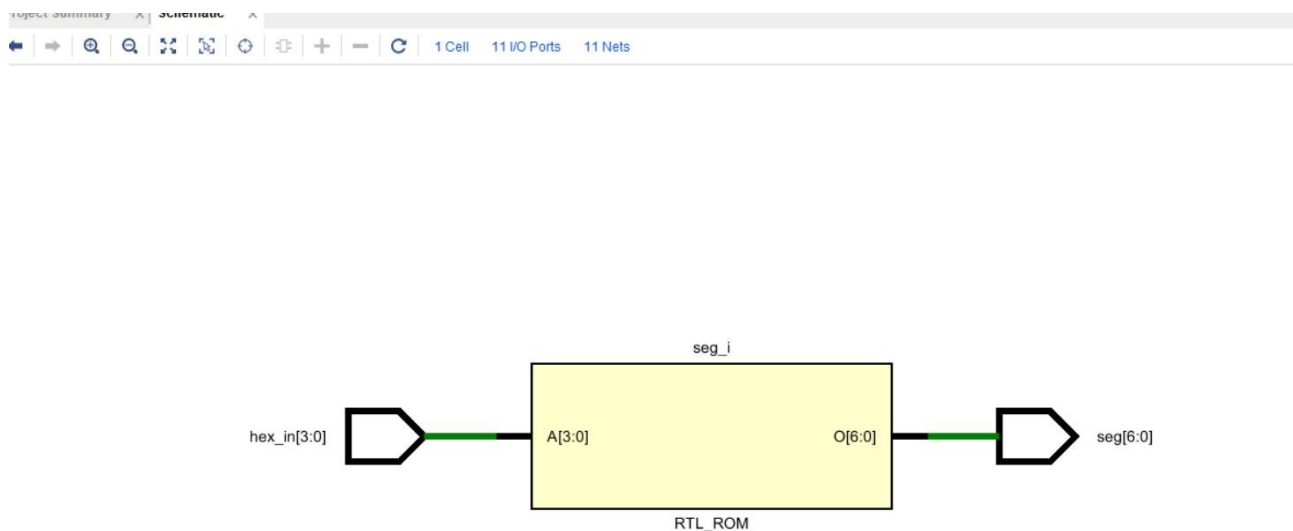
  report "TB_Display_Controller: Simulation done. Verify multiplexed
  Anode/Cathode in waveform." severity note;
  wait;
end process;

end Behavioral;

```

Hex to Seven Seg

Schematic Diagram



Design Source Code

```
-----
--
-- Module Name: Hex_to_7seg - Behavioral
-- Description: Converts a 4-bit hex value to 7-segment display segments.
--              Segments are ACTIVE LOW (standard on Basys3).
--
-- Segment mapping:
--      aaa
--      f   b
--      f   b
--      ggg
--      e   c
--      e   c
--      ddd      (dp not used)
--
--      seg = {g, f, e, d, c, b, a}  (seg[0]=a ... seg[6]=g)
-----
--
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Hex_to_7seg is
    Port ( hex_in : in  STD_LOGIC_VECTOR(3 downto 0);
          seg      : out STD_LOGIC_VECTOR(6 downto 0));  -- active low:
    {g,f,e,d,c,b,a}
end Hex_to_7seg;
```

```
architecture Behavioral of Hex_to_7seg is
begin
    process(hex_in)
    begin
        case hex_in is
            --
            --      gfedcba
            when "0000" => seg <= "1000000"; -- 0
            when "0001" => seg <= "1111001"; -- 1
            when "0010" => seg <= "0100100"; -- 2
            when "0011" => seg <= "0110000"; -- 3
            when "0100" => seg <= "0011001"; -- 4
            when "0101" => seg <= "0010010"; -- 5
            when "0110" => seg <= "0000010"; -- 6
            when "0111" => seg <= "1111000"; -- 7
            when "1000" => seg <= "0000000"; -- 8
            when "1001" => seg <= "0010000"; -- 9
            when "1010" => seg <= "0001000"; -- A
            when "1011" => seg <= "0000011"; -- b
            when "1100" => seg <= "1000110"; -- C
            when "1101" => seg <= "0100001"; -- d
            when "1110" => seg <= "0000110"; -- E
            when "1111" => seg <= "0001110"; -- F
            when others => seg <= "1111111"; -- all off
        end case;
    end process;
end Behavioral;
```

Simulation Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Hex_to_7seg is
end TB_Hex_to_7seg;

architecture Behavioral of TB_Hex_to_7seg is

    component Hex_to_7seg
        Port (
            Hex      : in  STD_LOGIC_VECTOR(3 downto 0);
            Seg_out   : out STD_LOGIC_VECTOR(6 downto 0)
        );
    end component;

    signal Hex      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Seg_out   : STD_LOGIC_VECTOR(6 downto 0);

begin

    UUT: Hex_to_7seg
        port map (Hex => Hex, Seg_out => Seg_out);

    process
    begin
        -- Test all 16 hex values and print expected segment patterns
        -- Segments: gfedcba (active low common anode assumed)
        Hex <= "0000"; wait for 20 ns; -- 0 => 1000000
        assert Seg_out = "1000000" report "FAIL: 0" severity error;

        Hex <= "0001"; wait for 20 ns; -- 1 => 1111001
        assert Seg_out = "1111001" report "FAIL: 1" severity error;

        Hex <= "0010"; wait for 20 ns; -- 2 => 0100100
        assert Seg_out = "0100100" report "FAIL: 2" severity error;

        Hex <= "0011"; wait for 20 ns; -- 3 => 0110000
        assert Seg_out = "0110000" report "FAIL: 3" severity error;

        Hex <= "0100"; wait for 20 ns; -- 4 => 0011001
        assert Seg_out = "0011001" report "FAIL: 4" severity error;

        Hex <= "0101"; wait for 20 ns; -- 5 => 0010010
        assert Seg_out = "0010010" report "FAIL: 5" severity error;

        Hex <= "0110"; wait for 20 ns; -- 6 => 0000010
        assert Seg_out = "0000010" report "FAIL: 6" severity error;

        Hex <= "0111"; wait for 20 ns; -- 7 => 1111000
        assert Seg_out = "1111000" report "FAIL: 7" severity error;

        Hex <= "1000"; wait for 20 ns; -- 8 => 0000000
        assert Seg_out = "0000000" report "FAIL: 8" severity error;

        Hex <= "1001"; wait for 20 ns; -- 9 => 0010000
        assert Seg_out = "0010000" report "FAIL: 9" severity error;

        Hex <= "1010"; wait for 20 ns; -- A => 0001000
        assert Seg_out = "0001000" report "FAIL: A" severity error;
```

```
Hex <= "1011"; wait for 20 ns; -- B => 0000011
assert Seg_out = "0000011" report "FAIL: B" severity error;

Hex <= "1100"; wait for 20 ns; -- C => 1000110
assert Seg_out = "1000110" report "FAIL: C" severity error;

Hex <= "1101"; wait for 20 ns; -- D => 0100001
assert Seg_out = "0100001" report "FAIL: D" severity error;

Hex <= "1110"; wait for 20 ns; -- E => 0000110
assert Seg_out = "0000110" report "FAIL: E" severity error;

Hex <= "1111"; wait for 20 ns; -- F => 0001110
assert Seg_out = "0001110" report "FAIL: F" severity error;

report "TB_Hex_to_7seg: All tests done." severity note;
wait;
end process;

end Behavioral;
```


7. Simulation Results

Each component was verified individually using a dedicated VHDL testbench before top-level integration. The top-level Nano_processor testbench applies a Reset pulse followed by a free-running clock and observes Result, Overflow, and Zero.

Component / Testbench	Key Observations
three_bit_adder	Sum correctly wraps on overflow; Cout asserted at boundary (e.g. $7+1=0$, Cout=1)
pc_incrementer	Output increments sequentially from 0 to 7 then wraps; Cout observed at wrap
Program_Counter	Reset holds Q=000; Q follows D on rising edge when Load='1'
ROM	Each address outputs the correct 12-bit machine code word; combinational, no clock delay
Instruction_decoder	All four opcodes produce correct control signal combinations; unused outputs default to 0
Registers_Bank	Selected register updates on clock edge; Reset clears all registers; non-selected registers unchanged
Add_Sub_unit	Addition and subtraction correct; Overflow asserted for out-of-range signed results; Zero asserted when result=0
Nano_processor (top-level)	R0=5, R1=3, R2=8, R3=-3, R4=5, R5=1, R6=8 observed in sequence; JZR at addr 5 does not branch as R0≠0

8. Problems Faced and Solutions

NEG instruction — forcing ALU input A to zero

Problem: The NEG instruction must compute $0 - R_s$. However, the ALU always reads its input A from a register selected by the MUX. There is no register hardwired to zero.

Solution: A Neg_En signal is asserted by the decoder for NEG instructions. A concurrent signal assignment in the top-level entity forces ALU_A to "0000" whenever Neg_En = '1', overriding the MUX output. The MUX_B input correctly selects the source register R_s .

JZR zero-check performed on Rs, not ALU result

Problem: The JZR instruction must check whether register Rs is zero before branching. The ALU Zero flag reflects the result of an arithmetic operation, which is not the same as checking a register's stored value directly.

Solution: A separate Zero_check signal is derived combinatorially from ALU_A_raw (the raw MUX-A output before the Neg_En override). Zero_check = '1' when ALU_A_raw = "0000". This signal is ANDed with the Jump flag to produce Jump_en, which controls the PC MUX.

Opcode field width and instruction encoding

Problem: Aligning a clean encoding across four instructions while keeping the register field and immediate fields non-overlapping required careful bit-field planning.

Solution: The opcode occupies bits [11:9] (3 bits, supporting up to 8 instructions), Rd occupies bits [8:6], source registers Rs1/Rs2 or the immediate value occupy bits [5:0]. This separation ensures the decoder logic is straightforward and extensible.

9. Conclusion

This project successfully delivered a fully functional 4-bit nanoproessor implemented in VHDL. The processor correctly executes the MOVI, ADD, NEG, and JZR instruction set, and was verified through both component-level simulation and full system simulation. Key design decisions — including the structural hierarchy, the Neg_En override for zero-operand subtraction, and the combinational Zero_check for conditional branching — were validated during simulation.

Working through this project reinforced practical skills in hierarchical VHDL design, structural instantiation, control-path and data-path separation, and systematic testbench development. The modular structure also made debugging straightforward: faults in individual components were isolated and corrected before integration, significantly reducing top-level debugging effort.

10. Team Contribution

Team Member	Contribution	Hours
240403E	Mux-2-3, Mux-8-4bit, Mux 2-4bit, Mux 8-4bit new, Comparator	7
240439T	Instruction decoder, Program counter, Rom, Hex to Seven Seg, Display Controller	8
240426C	Tri state buffer 4 bit, Tri state buffer, Reg 4 bit, Register bank, Seven-seg-decoder	7.5
240441R	Nano processor, Slow clock, Three bit adder, Add sub unit, Pc incrementor	7.5
All Together	Lab report, Debugging, Test on the board, Assembling nano processor	8